

江西理工大學

本科毕业论文(设计)

题　目：“图书馆智能预约占座系统”微信小程序开发

专题题目：

学　院：信息工程学院

专　业：计算机科学与技术

班　级：计算机 224 班

学　号：　　学　生：程鹏

指导教师：谢润山　职　称：讲师

指导教师：　　职　称：

时　间：2026 年　6 月

摘　要

高校图书馆在考试周、考研复习等高峰阶段容易出现座位资源紧张、占座现象突出以及人工管理效率不足等问题。针对上述情况，本文设计并实现了一套图书馆智能预约占座系统。系统以微信小程序作为读者端入口，后端基于 Node.js 提供接口服务，并使用 MySQL 存储用户、分区、座位、预约、签到、暂离、信用和公告等数据，实现座位查询、预约校验、预约创建、签到签退、暂离返座、记录查询、规则配置和后台管理等功能。在智能推荐方面，本文将自然语言座位需求与座位空间属性相结合，把推荐任务建模为需求与座位之间的多标签匹配问题。实验利用 56 条代表性用户需求和 1772 个候选座位生成 99232 条匹配样本，经过筛选与类别均衡处理后保留 34216 条样本用于训练和测试，并提取 68 项输入特征，设置深度专注、舒适久学、设备作业和开放协作四类输出标签。实验结果表明，支持向量机 SVM 取得较好的综合表现，Macro-F1 为 0.9065，Micro-F1 为 0.9269，Top-5 推荐精度达到 0.9636。系统测试结果表明，本文系统能够支撑图书馆座位预约管理的主要业务流程，可为高校图书馆座位服务的数字化和智能化建设提供参考。

关键词：微信小程序；图书馆座位智能预约；机器学习；多标签分类

ABSTRACT

University libraries often face seat shortages, prominent seat-occupying behavior, and insufficient manual management efficiency during peak periods such as examination weeks, postgraduate entrance examination preparation. To address these issues, this thesis designs and implements an intelligent library seat reservation and occupancy management system. The system uses a WeChat Mini Program as the reader-side entrance, provides backend services based on Node.js, and stores data such as users, zones, seats, reservations, check-ins, temporary leaves, credit records, and announcements through MySQL. It supports seat inquiry, reservation validation, reservation creation, check-in and check-out, temporary leave and return, record inquiry, rule configuration, and backend management. In terms of intelligent recommendation, this thesis combines natural language seat requirements with spatial seat attributes and formulates the recommendation task as a multi-label matching problem between user requirements and candidate seats. The experiment generates 99,232 matching samples from 56 representative user requirements and 1,772 candidate seats. After filtering and class balancing, 34,216 samples are retained for training and testing. A total of 68 input features are extracted, and four output labels are defined: deep focus, comfortable long-term study, device-based work, and open collaboration. The experimental results show that the Support Vector Machine (SVM) achieves strong overall performance, with a Macro-F1 of 0.9065, a Micro-F1 of 0.9269, and a Top-5 recommendation precision of 0.9636. System testing results indicate that the proposed system can support the main business processes of library seat reservation management and provide a reference for the digital and intelligent development of university library seat services.

Keywords: WeChat Mini Program; Intelligent Library Seat Reservation; Machine Learning; Multi-label Classification

目　录

第一章　绪论 1

1.1 研究背景与意义 1

1.2 国内外研究现状 2

1.3 本文研究内容 3

1.4 本章小结 4

第二章　开发技术与理论基础 5

2.1 微信小程序技术 5

2.2 Node.js 后端与管理后台 5

2.3 MySQL 数据库与数据组织方式 5

2.4 支持向量机与多标签分类基础 6

2.5 常用分类算法与模型评价指标 6

2.6 本章小结 7

第三章　需求分析 8

3.1 业务背景分析 8

3.2 功能需求分析 8

3.3 非功能需求分析 11

3.4 可行性分析 12

3.5 本章小结 12

第四章　系统设计 13

4.1 系统技术架构 13

4.2 系统总体架构设计 14
4.3 数据库设计 15
4.4 预约业务流程设计 19
4.5 智能推荐流程设计 20
4.6 本章小结 22
第五章 系统实现与算法设计 23
5.1 小程序端实现 23
5.2 后端业务实现 33
5.3 管理后台实现 33
5.4 需求-座位多标签匹配数据集构建 40
5.5 匹配标签生成与特征构造方法 42
5.6 多标签分类模型训练与在线推荐接入 43
5.7 本章小结 45
第六章 系统测试与实验分析 46
6.1 测试环境与评价指标 46
6.2 功能测试 46
6.3 业务规则与访问控制测试 49
6.4 性能与并发测试 50
6.5 推荐算法实验结果与分析 52
6.6 本章小结 59
总 结 61

参考文献 62
致　谢 64

第一章　绪论

1.1 研究背景与意义

高校图书馆座位资源紧张的情况并非一直均匀出现，而是主要在考试周、考研复习和毕业设计这些学习任务比较集中的时间段发生，读者到馆后既要寻找空座，也要判断座位周边是否安静，是否便于充电，并且是否适合进行长时间学习，不过传统人工巡查方式虽然可以处理明显占座情况，但是很难及时对预约、签到、暂离和违约等状态改变进行记录。魏鲁原和姜雨辰对高校图书馆座位管理成本进行过研究[1]，本文把座位状态感知、预约冲突校验和规则执行效率当作系统设计的主要问题来源。

实际业务表明，图书馆预约系统不能只停留在登记一条预约记录的方面上，读者在进行预约前应当先了解开放规则、余位情况、座位环境，而在完成预约之后也必须获取签到、暂离、返座、签退等状态反馈，管理员需要根据预约数据来判断座位利用率、爽约情况和规则执行效果，所以，本文把读者端操作、后台治理和信用记录放到同一个业务链条里，让一次预约可以继续关联后续履约状态和异常处理结果。

微信小程序更适合在移动端入口进行选择，因为它可以承载图书馆座位预约这类轻量、高频并且带有校园场景特征的服务，在已有的研究中，李昊楠和刘雅莉完成了对空闲座位的查看、预约信息的管理[2]，潘俊芳和侯振兴把登录、预约、签到、离座这些功能整合到了统一的流程里[3]，这些工作说明小程序可以减少读者使用的难度，这也提示本文在设计读者端时应当把座位查询、预约创建、状态反馈和后台规则维护进行连接。

普通预约规则主要说明座位在特定时间段内是否可以预约，但是读者在实际选座时也会进一步判断座位是否适合当前的学习任务，比如，同样处于空闲状态的座位中，靠窗并且有插座的位置更适合长时间进行论文写作或者设备学习，而靠近通道的位置可能会不利于进行专注复习，鉴于该差异，本文除了进行可预约性判断外，还引入需求和座位属性的匹配建模，使系统可以从可用座位筛选扩展到适配座位推荐。

围绕“图书馆智能预约占座系统”微信小程序开发这一课题，本文先实现读者预约、签到签退、暂离返座以及后台规则维护等基础业务流程，再将 AI 辅助选座作为扩展功能接入。推荐模块以读者输入的需求文本、预约日期和时间窗为起点，在过滤不可预约座位后，结合座位环境属性、桌组统计、分区统计、偏好命中和冲突项构造 68 项匹配特征，并调用对比实验中表现较好的 SVM 模型生成推荐结果。

1.2 国内外研究现状

从国内研究看，图书馆预约占座系统的相关工作主要围绕预约流程优化、业务流程和后台管理展开。向昌盛从小程序预约业务出发，设计了用户注册、网上预约、座位管理和信息公告等核心模块[4]；梁浩等进一步将登录、预约、签到、签退和管理员管理统一到完整业务链路中[5]。这些成果说明，图书馆座位预约已经由单纯的登记功能发展为面向规则执行和服务治理的业务系统，系统设计也逐渐从“能预约”转向“可管理、可追踪、可治理”。

国外有关研究主要分析座位管理系统共同运行的能力，Jing 等对预约、选座和座位状态维护进行了座位管理流程分析[6]，Guo 等将数据采集、座位分配和监控功能归入智能座位管理模式[7]，Xu 等则从智慧图书馆空间服务方面分析了座位管理系统所发挥的作用[8]，这些研究为本文的预约状态管理提供了参考，但是面向读者即时学习需求的座位匹配和推荐还有进一步展开的空间。

在推荐系统研究方面，推荐结果并不只取决于资源是否可用，还需要综合考虑用户偏好、项目属性和上下文条件之间的匹配关系。Adomavicius 和 Tuzhilin 对推荐系统研究进行了系统梳理，指出推荐系统的核心在于建模用户需求与项目特征之间的关系[9]。对于图书馆座位预约场景而言，座位的安静程度、是否靠窗、是否具备插座、所在分区、预约时段和当前占用状态等因素，都可能影响读者的实际选择。因此，将推荐思想引入座位预约系统，能够使系统从单纯的规则筛选进一步扩展为面向学习场景的辅助决策。

本文进行的座位推荐任务并不是对座位本身的好坏进行单独评价，主要在于判断读者的实际需求和候选座位之间的匹配程度，只收集座位的安静度、是否靠窗、有没有供电插座等基础属性，还不足以支持个性化推荐的实现，系统还需要进一步判断这些属性是不是符合当前用户的具体要求，根据该情况，接下来形成的模型把相关特征分成需求端特征、座位端特征、两者交叉匹配特征三类，并且使用多标签分类方法，同时输出深度专注、舒适长时学习、设备操作作业、开放协作四种场景的适配结果。

通过梳理已有文献可以看出，现阶段图书馆预约系统的研究还有三个方面的欠缺：第一，业务系统分析和推荐算法分析大都各自分开，没有把真实预约规则、座位情况和用户要求进行统一设计，其次，现有系统主要关注可预约查询和规则管理，较少对自然语言需求解析、候选座位筛选、多标签匹配预测、推荐排序返回等完整链路进行分析，第三，微信小程序端、后台管理端、数据集建立、模型实验结果之间衔接方式仍然不够充分。针对上述问题，本文计划在系统实现和算法设计这两个方面进行一体化研究。

1.3 本文研究内容

1.3.1 研究目标

本文主要目标是在高校图书馆场景下设计并实现一个拥有预约全流程、后台治理和智能推荐能力的微信小程序系统，本文希望读者可以利用移动端进行分区查看、条件筛选、预约提交、签到签退、暂离返座和记录查询等操作，管理员也能够通过后台完成公告发布、预约处理、规则配置、分区维护和座位管理等工作，并在此基础之上通过 AI 推荐入口把训练后的分类模型接入实际选座流程。

本文将算法目标分为候选座位筛选和候选座位排序这两个阶段，系统先对用户输入进行解析，从中找出安静、靠窗、插座、采光、多人学习等偏好，并且结合预约日期、时间窗、座位状态和时隙占用情况来形成可预约的候选集合，后端接着将同一条需求和候选座位进行组合，从而形成 68 项需求与座位匹配特征，通过 SVM 模型计算出四类匹配标签的概率，随后按照偏好命中和冲突惩罚规则生成 Top-N 推荐列表。

1.3.2 研究内容

围绕上述研究目标，本文的主要研究内容包括以下四个方面。

本文第一部分工作是整理图书馆座位预约管控的主要业务需求，在普通读者使用端、管理人员运维端和系统运行规则层这三个方面，对预约发起、到馆签到、临时离座、回座核验、使用结束签退、预约取消、违约判定处置等全过程节点进行拆解，并且说明各操作环节对应的用户状态、座位使用状态、预约流水记录和信用积分数据的读取与更新规则。

第二部分工作是完成系统总体设计、工程实现方案设计，本文主要围绕微信小程序读者端、Node.js 后端接口、Web 管理后台和 MySQL 数据库这四个方面进行，依次说明页面功能、接口调用、后台治理、数据表之间的组织关系，并且结合预约、签到、暂离、信用记录和推荐结果等重要对象，确定系统后续实现需要的模块边界和数据联系。

第三部分的工作是形成用于推荐实验的需求座位多标签匹配数据集，本文利用 1772 个座位主数据和 56 条典型需求文本，采用两两组合的方法生成 99232 条需求座位样本，本文对样本进行了筛选和均衡处理，保留了 34216 条样本，用来进行训练和测试，每个样本包括 68 项特征，涉及需求偏好、座位属性、桌组和分区统计、偏好命中、冲突项、场景评分，并且与四个多标签输出相对应。

第四部分工作是完成模型对比实验和在线推荐接入。实验统一使用 `features.xlsx`、`labels.xlsx` 和 `output.mat`，比较逻辑回归、SVM、MLP、随机森林、决策树、K 近邻和高斯朴素贝叶斯等模型；训练测试划分按需求编号分

组，并分别设置 8:2、7:3 和 6:4 三种比例。综合 Macro-F1、Micro-F1、子集准确率和召回率等指标后，本文选择 SVM 接入小程序 AI 辅助选座接口。

1.4 本章小结

本章主要针对高校图书馆高峰时段座位供需失衡、占座行为管控、预约规则落地等问题进行分析，并且结合国内和国外学界在预约系统搭建、小程序功能优化、智能推荐算法方面已经取得的研究成果，确定了本文的两个主要任务：第一个是形成包括读者操作端和管理后台的全流程预约业务流程，第二个是把读者的自然语言使用需求与座位自身属性特征变成可以训练的多标签匹配任务，后续章节会以此为基础，依次对技术选型依据、需求分析结果、系统架构设计、功能模块实现、推荐模型建立和系统测试结论进行详细说明。

第二章　开发技术与理论基础

2.1 微信小程序技术

微信小程序是依靠微信生态运行的轻量级应用形态，用户无需单独安装客户端，便可以通过微信入口进入系统并进行操作，小程序页面主要利用 WXML 组织结构，通过 WXSS 完成界面样式，依靠 JavaScript 处理页面状态、接口请求和交互逻辑，开发方法和 Web 前端有相似之处，也可以较好地适配移动端的高频业务场景。

本文将微信小程序视为读者端的接入载体，主要由于读者的操作主要在移动场景中进行，涉及公告查阅、到馆时段选择、阅览分区浏览、预约申请提交、入馆签到或离馆签退、临时离开后返座、个人使用记录查询等功能需求，小程序可以把首页展示、座位选择、预约结果确认、智能辅助选座和个人中心等模块整合到同一个入口，并且通过服务接口和后端系统进行数据交互，能够适配图书馆预约业务流程短、触发频次高的运营特点。

2.2 Node.js 后端与管理后台

Node.js 在本文系统中主要承担接口服务和业务规则组织的作用。小程序端与管理后台的请求都需要经过后端完成参数检查、规则判断和数据库访问，例如座位查询、预约预校验、签到签退、公告发布和规则读取等。由于前端页面也主要使用 JavaScript 编写，后端继续采用 Node.js 能够减少开发过程中的语言切换，并提高调试和联调效率。

本文设计的系统后端接口可以同时为微信小程序读者终端和 Web 管理后台提供服务支持，读者终端能够调用的接口有用户登录、座位状态查询、预约资格预校验、预约订单生成、签到签退、暂离后返座核验、AI 智能选座等功能模块，管理后台具备发布公告、配置使用规则、维护座位信息、调整用户信用分、统计分析运营数据等功能，并且这些功能均依靠接口来进行提供。Node.js 会对所有接口进行统一的调度管理，预约规则判定、座位状态同步和选座推荐结果输出等主要逻辑都在服务层进行维护，这样做是为了防止前后端各自实现校验逻辑所产生的规则不一致问题。

2.3 MySQL 数据库与数据组织方式

MySQL 目前被广泛使用，作为开源关系型数据库管理系统，它具有运行稳定性高、部署容易、查询响应快等特点，关系型数据库将二维数据表作为信息存储的基本单元，通过主键、外键、索引、事务等方式来保证数据关联的一致性和完整性，非常适合结构化程度高、数据之间关联逻辑清晰的业务场景的数据存储

需要，本文所形成的系统数据库需要同时达到对静态资源数据和动态业务数据这两种信息进行存储的要求。用户、分区、座位、桌组是变动频率很低的静态资源数据，预约、时隙、签到、暂离、信用日志、公告会业务会业务的持续进行而进行实时更新，单条预约记录把用户主体、对应座位、预约日期、使用时段、后续签到状态等各方面信息联系起来，MySQL 通过主键外键约束、索引优化和事务控制等功能，可以对这些复杂的关联关系进行有效管理，并且能够很好地支持预约冲突校验和业务状态流转记录等功能的实现。

2.4 支持向量机与多标签分类基础

本文把支持向量机用来处理结构化的需求座位匹配特征，推荐样本不单是座位或需求，而是需求文本和候选座位组合而成，模型要依据 68 项特征判断该组合在不同学习场景下的适配情况，因为本文样本规模适中，并且特征大多属于明确的数值或者布尔属性，所以 SVM 适合用作多模型对比实验中的主要候选模型。

多标签分类与普通单标签分类有所不同，一个样本可以同时对应多个标签。本文中的样本不是单独的用户需求，也不是单独的座位，而是由“一条用户座位使用需求”和“一个候选座位”组合形成的需求-座位匹配样本。在数据构造时，系统将每条用户需求分别与候选座位逐一配对，每一对需求和座位形成一条样本。因此，样本数量由需求数量和候选座位数量共同决定，而不是由标签数量决定。

本文给所有需求座位样本设定了四种输出标签，分别为深度专注匹配、舒适久学匹配、设备作业匹配和开放协作匹配，用户若同时提出环境安静、可用充电接口、长时间学习这三种需求，单个样本就会同时匹配多个标签，如果候选座位同时装有充电插座、位于低噪音区域并且距离人流通道较远，该样本会被归入专注、舒适和设备作业这三个类别。采用多标签标注方法能够完整保存此类场景的重叠联系，在进行座位排序时输出粒度比较细的概率参考值。

2.5 常用分类算法与模型评价指标

分类算法主要根据样本特征来判断其所属类别，是机器学习中比较重要的方法，本文没有单纯依靠经验选择推荐模型，而是把逻辑回归、支持向量机、神经网络 MLP、随机森林、决策树、K 近邻和高斯朴素贝叶斯放在一起进行统一的实验对比，并且在相同的需求座位多标签匹配数据集上进行测试，处理完成之后，各个模型使用相同的 features.xlsx、labels.xlsx 和 output.mat 文件，实验结果可以反映出该算法在当前数据条件下的适用情况。

不同分类算法的侧重点并不相同。随机森林通过多棵决策树进行集成，可以缓解单棵树结果波动的问题[10]。支持向量机通过间隔最大化思想寻找分类边界，适合结构化特征较多且样本规模适中的分类任务[11]。K 近邻依据样本距离进行

判断，思想直观，不过在候选样本较多时预测成本会增加[12]。决策树规则清楚，便于解释，但对数据扰动较敏感[13]。神经网络 MLP 能够通过误差反向传播学习特征之间的非线性关系[14]。TabNet 针对表格数据建模引入注意力机制，能够在一定程度上提升特征选择和结果解释能力[15]。Gorishniy 等关于表格数据深度学习模型的研究说明，不同深度模型在结构化数据上的表现会受到数据规模和特征分布影响[16]。Borisov 等对表格数据深度神经网络进行了综述，指出表格数据建模仍需要结合任务特点选择合适模型[17]。Grinsztajn 等的研究表明，在典型表格数据任务中，树模型在不少场景下仍具有较强竞争力[18]。因此，本文没有预设单一最优模型，而是在同一需求-座位匹配数据集上进行多模型对比。

模型评价不可以只依靠单一的准确率，由于本文的样本同时包括四个标签，子集准确率需要四个标签都预测正确，评价标准比较严格，Macro-F1 分别计算各个标签的表现，然后进行平均，这样可以方便地观察开放协作等相对困难的标签是否被忽略，Micro-F1 从全部样本和标签的整体方面统计预测效果，所以，后文在选择部署模型时主要参考多项指标之间的平衡关系。

2.6 本章小结

本章主要说明系统开发和推荐实验所依靠的技术基础，微信小程序可以用于读者端的移动预约操作，Node.js 负责对小程序和后台管理端的接口请求进行统一处理，MySQL 主要用来存储用户、座位、预约、信用、公告和规则这些结构化数据，在推荐算法上，本文将座位推荐问题转化为需求和座位多标签匹配任务，并且说明了 SVM、多标签分类、常用对比模型和评价指标。前面所述内容为接下来进行系统架构设计、数据库组织、接口实现和模型实验提供了技术依据。

第三章　需求分析

3.1 业务背景分析

图书馆座位的使用表现出比较突出的高峰时段特点，在考试周、考研复习、毕业设计集中进行的阶段，读者到馆之后经常会出现空座难以找到、物品占据座位、座位状态不清楚等情况，仅通过现场巡查很难持续记录预约、签到、暂离和违约情况，在该业务背景下，本文应当设计一个可以记录座位状态、校验预约冲突并且保留履约过程的预约系统。

本文结合图书馆座位使用场景，认为系统应当重点考虑三个方面的业务特点，座位需求表现出时段集中性，不同日期和时间段的座位压力不一样，系统可以按照日期和时间段查询可用座位，并且在预约时进行冲突校验，座位资源在空间上存在差异，各分区、桌组和座位在安静程度、插座、靠窗、通道位置等方面有所不同，读者选座时不仅关心有无空位，也关注座位是否满足学习需求。第三点是座位管理具有规则约束性，预约、签到、签退、暂离、取消和违约处理都需要按统一规则执行，否则会出现重复预约、长期占座或后台难以追踪的问题。

本系统使用的座位主数据包括 1772 个座位、54 个分区和 291 个桌组，其中 8 人桌座位有 1216 个、4 人桌座位有 556 个，拥有插座的座位有 1380 个，靠窗座位有 484 个，安静座位有 1556 个，采光较好的座位有 1380 个，这些字段用于页面展示，并且参与分区筛选、预约可用性判断和推荐特征构造，所以座位主数据是普通预约和 AI 选座共同依赖的数据基础。

本文整体目标包括读者端预约、管理员端治理和智能推荐这三个方面，读者端主要负责进行移动预约和履约反馈，管理员端则承担维护座位、公告、规则和异常预约的任务，推荐模块会在可预约座位范围内给出更符合需求的候选结果，系统要实现上述目标，就需要同时保存座位的静态属性、预约的动态状态、规则的配置、读者的需求，使读者的请求、后台的维护和推荐的排序都可以依靠同一套数据来进行运行。

3.2 功能需求分析

依据系统服务对象、业务流程和推荐链路，本文把功能需求划分成读者端功能、管理员端功能和智能推荐功能三个部分，读者端主要关注预约与使用过程，管理员端主要关注资源维护和规则治理，智能推荐功能主要关注把自然语言需求转化为可以预约候选座位的匹配排序结果。

读者端功能要可以拥有对单次预约进行完整流程的功能，用户打开小程序后，会先进行账号登录，接着查看首页信息，然后选择学习偏好入口，浏览功能分区，筛选座位条件，进行预约前校验，创建预约订单，查询当前预约，进行签到和签

退，在暂离后返回座位，调取历史记录，预约预校验属于主要环节，系统在正式把预约信息录入数据库之前，会对用户账号状态、拟预约时段、座位当前状态、对应时隙占用情况、平台预约规则约束等内容进行逐一检查。

管理员端的功能主要针对图书馆的日常管理，管理员应当拥有维护公告、用户、分区、座位、预约记录和系统规则的能力，并且可以查看运营统计信息，当发生异常预约、用户违约或座位维护等情况时，管理员可以经由后台进行相应处理，包括释放异常预约、调整用户信用状态、停用或恢复座位等，后台管理可以利用这些功能，对前台预约流程进行支撑。

智能推荐功能主要服务于有明确学习偏好的读者，用户在输入安静、靠窗、可以充电、适合长时间学习等需求后，系统会识别插座、靠窗、安静、采光、远离通道、多人学习等偏好，并且将日期、时段、座位状态和时隙占用进行结合，对候选座位进行筛选，后端构造需求和座位匹配特征，利用 SVM 输出四类标签概率，并按照偏好命中和冲突项形成排序结果。

表 3-1 系统功能模块划分

模块名称	主要功能	主要使用者
首页概览	展示开放分区、公告信息、热点区域与快捷入口	普通读者
分区与选座	浏览分区、筛选座位属性、查看桌组与座位状态	普通读者
预约管理	预约预校验、创建预约、取消预约、查看预约	普通读者
签到与信用	签到、签退、暂离返座、查询信用分与预约记录	普通读者
后台管理	公告、规则、分区、座位、用户与预约记录维护	管理员
智能推荐	偏好解析、候选座位筛选、场景分类预测与推荐	普通读者/管理员

由表 3-1 可知，系统功能主要围绕读者预约、后台管理和智能推荐这三个方面进行进行展开，读者端主要对座位查询、条件筛选、预约提交、签到签退和记录查看等方面的问题进行解决，管理员端主要进行公告维护、预约处理、规则配置、分区维护和座位管理等任务，智能推荐功能会在可以预约的座位范围内，根据读者需求和座位属性来生成推荐结果。三类功能之间存在数据关联，后台维护的规则和座位状态会影响到前台预约结果，读者进行的预约、签到和暂离行为会形成后台治理数据，推荐结果需要经过预约时间、座位状态和用户资格等规则校验后，才可以进入最终预约流程。

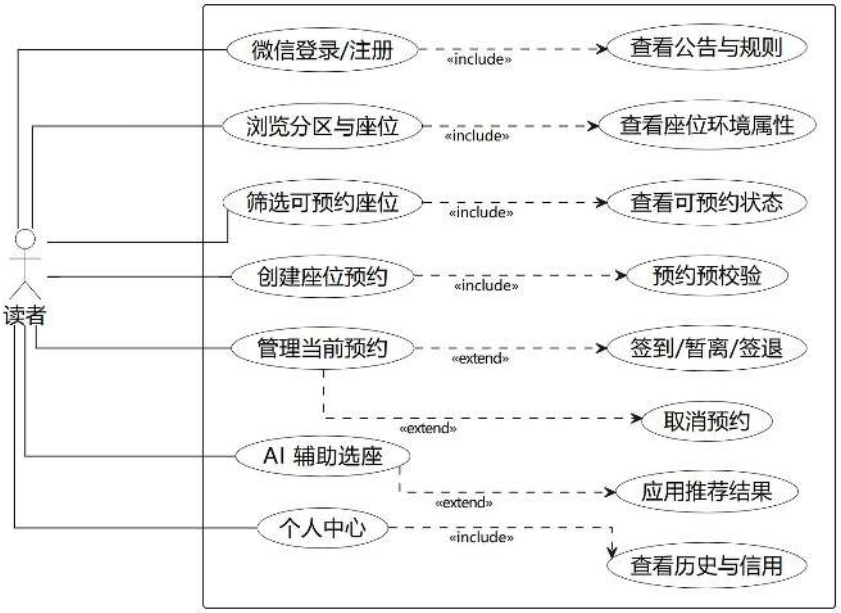


图 3.1 读者端功能用例图

图 3.1 从读者侧归纳了系统需要支撑的主要功能，读者在完成登录操作后，能够进入分区浏览页面、座位筛选页面、预约创建页面、当前预约页面、AI 辅助选座页面、个人中心页面，系统会在用户进行预约之前显示公告、规则、座位属性和指定时间窗所对应的可预约状态，预约开始时，系统会对用户的资格、时间、座位状态和规则限制进行预校验。预约完成后，读者可以在当前的预约页面里继续进行签到、暂离、返座、签退或者取消操作，AI 辅助选座将自然语言需求转变成可以计算的偏好，并且在可预约座位里返回排名、匹配度、标签和推荐理由。

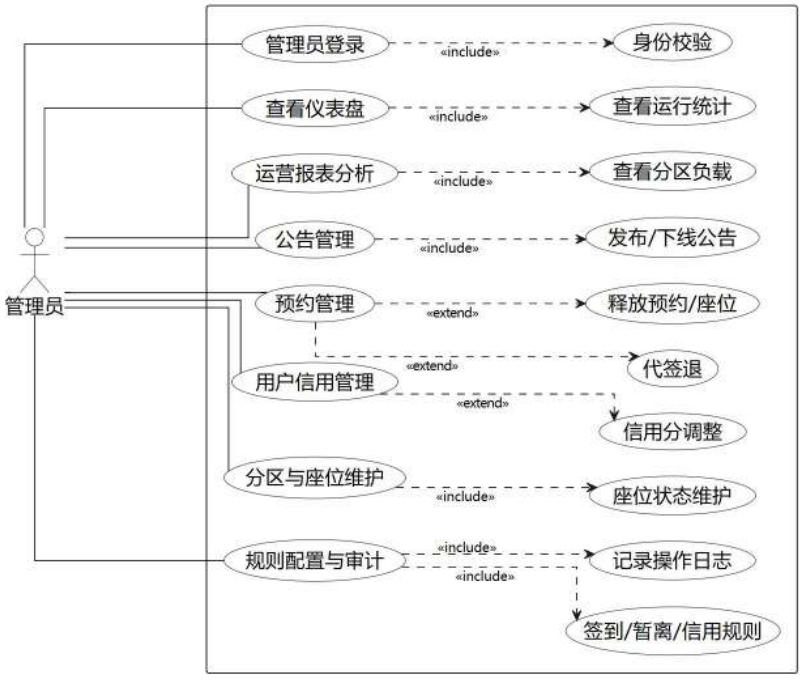


图 3.2 管理员功能用例图

图 3.2 显示了管理员侧的主要功能，管理员在登录后台之后，可以查看运行统计和运营报表，也可以对公告、预约、用户信用、分区座位、系统规则进行维护工作，预约管理功能可以处理异常预约和代签退情况，用户管理功能可以对信用分进行调整，座位管理功能可以进行分区编辑、座位属性维护、状态变更，规则配置功能主要用于维护签到、暂离和信用参数。后台操作会影响到小程序端公告、座位可预约状态和预约预校验结果。

3.3 非功能需求分析

系统除了满足基本功能外，还需要在易用性、性能、可靠性、安全性和可维护性等方面达到一定要求。从软件工程角度看，需求分析阶段应确定系统的功能需求、性能需求、可靠性和可用性需求等综合要求[19]。GB/T 25000.10-2016 将功能适合性、性能效率、易用性、可靠性、安全性和可维护性等作为软件质量评价的重要维度[20]。对于读者端而言，页面流程应当尽量清晰，座位筛选、预约提交、AI 推荐结果查看和状态反馈等操作要便于理解，系统在预约成功、预约失败、签到异常、暂离超时等情况下也要给出明确提示；对于管理员端而言，后台页面需要支持列表筛选、数据查看、规则维护和异常处理，使管理员能够较快定位预约问题、用户信用变化和座位资源状态。

查询、预约和推荐这些环节表现出性能和可靠性要求，座位列表、预约记录、后台统计页面应该在较短的时间内返回，预约创建需要防止同一座位在同一时段被重复占用，签到、暂离、签退和信用日志也应当和预约状态保持对应，AI 选

座接口要先对无法预约的座位进行过滤，然后进行模型预测，这样可以防止推荐结果同真实预约规则出现冲突。

系统安全架构的主要基础是实现读者和管理员两种角色的权限分级管理，普通读者只能使用个人预约查询、座位检索、智能选座、个人中心等面向用户的常用功能，后台公告推送、阅览规则设置、座位信息维护、用户信用分值调整等后端操作接口应当设定为管理员专有权限，用户在提交预约申请、取消已有的预约，或者管理员进行信用等级调整、阅览规则修改这类主要操作时，系统会先对请求包括的各个参数进行合法性校验，并且把所有主要操作的日志信息完整保存归档，为后续的问题溯源和排查工作提供可靠的数据依据。

在可维护上，系统应保持小程序页面、Web 管理后台、后端接口、数据库结构和推荐模块之间的边界清晰。预约规则、公告内容、座位状态、环境标签和推荐策略都可能随着图书馆管理要求变化而调整，因此系统需要保留配置和扩展空间，使后续能够在不大幅改动主体业务流程的情况下，继续补充新的座位属性、预约规则或模型数据。

3.4 可行性分析

3.4.1 技术可行性

从技术条件看，本课题所需组件已经能够支撑原型系统实现。读者端采用微信小程序，后端使用 Node.js 提供接口，MySQL 保存用户、座位、预约和后台数据；推荐实验的数据已整理为 `features.xlsx`、`labels.xlsx` 和 `output.mat`，并完成七类模型对比。当前后端通过 `/api/assistant/seat` 调用 SVM 实时预测脚本，将自然语言需求、时间窗和候选座位转化为推荐结果，因此系统开发和算法接入在技术上具有可行性。

3.4.2 经济可行性

该系统一期建设主要依靠软件开发环境和普通服务器资源，不需要额外购买大量专用硬件，读者端在微信小程序里运行，管理端利用 Web 页面进行访问，接口服务和数据库可以在常规服务器环境中进行部署，未来若想提高签到准确度或座位识别功能，本文认为现有系统能够逐步接入扫码、定位、门禁记录等数据源，不用在初期就投入复杂的硬件设备。

3.4.3 运行可行性

就系统运行逻辑来说，读者和管理员这两种用户的权责划分非常清楚，读者主要进行座位查询、使用预约、到馆签到、临时离场登记、使用签退、异常情况反馈等操作，管理员则负责公告发布、使用规则制定、座位资源统筹、异常预约

处置等工作，推荐模块仅会在可以预约的候选座位池里进行优先级排序，系统会对静态座位属性、动态预约记录、后台运营数据、算法输出结果分别进行分类存储。对预约规则进行调整或扩大推荐策略时不需要对整体业务逻辑进行重构。

3.5 本章小结

本章以高校图书馆座位使用场景为基础，对系统需求进行了整理，座位预约业务不仅有查询空位和提交预约的功能，而且在高峰时段会出现资源紧张的情况，不同分区环境存在差异，签到暂离规则需要执行，违约记录要被追踪，后台人工也会进行干预，结合这些特点，本文将需求划分为读者端预约、管理员端治理和 AI 辅助选座三种类型：读者端主要关注查询、筛选、预约和履约反馈，管理员端重视公告、规则、座位和信用维护，推荐模块会于可预约的候选座位范围内进行需求解析和匹配排序。本章也从易用性、性能可靠性、安全性和可维护性方面提出了非功能要求，并从技术、经济和运行角度说明了课题具备实现条件，由此为后续系统设计和算法实现提供了依据。

第四章　系统设计

4.1 系统技术架构

本文根据之前完成的用户需求分析结果，把系统整体划分成读者交互端、管理后台、业务服务层、数据存储层和算法模块五个主要部分，微信小程序端负责进行读者预约提交和 AI 智能选座，Web 管理后台主要用于发布公告、设置使用规则、管理座位资源和维护用户信用信息，Node.js 服务对全平台的接口调用请求进行统一响应处理，MySQL 数据库存储用户信息、座位数据、预约记录、签到信息、信用分值和系统规则等重要数据，算法模块依次进行需求解析、用户和场景特征构造、SVM 模型预测、推荐结果排序输出。该模块化架构设计让各功能可以单独进行开发和调试，并且方便后续系统迭代维护工作。

微信小程序主要面向普通读者，该应用具备首页、分区浏览、座位筛选、预约提交、签到签退、个人中心、AI 辅助选座等功能，Web 管理后台主要供管理员使用，管理员能够发布公告、处理预约、管理信用、维护座位分区、配置规则，并且进行运营统计，两个入口服务对象存在差异，但它们都访问相同后端接口和数据库数据，后台在对座位维护状态进行修改之后，小程序选座页面和预约预校验页面也会受到同步影响。

后端服务的工作是对数据进行增加、删除、修改和查询，并且将业务规则落实到接口层上，系统在建立预约之前，会检查用户身份、预约时间、座位状态、同账号冲突和同座位冲突，预约提交完成以后，预约主表和时隙记录要同步写入，接着进行的签到、暂离、签退、取消操作也会继续更新状态，系统通过接口层和数据库层这两种约束，可以减少同一座位在同一时间段出现重复预约的情况。

算法模块只对符合预约规则的候选座位进行运算处理，用户在提交个性化需求后，系统会对用户在插座配置、靠窗位置、环境安静度、采光条件、多人结伴学习等方面所具有的偏好进行分析，并且整合候选座位本身的属性、所属桌组的关联信息、场馆分区的统计数据，以生成匹配特征向量，支持向量机输出四类标签对应的概率之后，后端会结合偏好命中情况、冲突项校验结果和业务运营规则来计算座位综合得分，这种方式得到的推荐结果可以直接接入实际预约链路，不会局限于离线实验场景下的效果展示。

在数据流上，读者提交日期、时间段、座位条件或需求文本后，后端读取规则配置、座位主数据和预约占用记录，从而生成可以进行预约的候选集合，用户使用 AI 选座功能时，系统会对这些候选座位产生运行时特征，并且进行批量预测，管理员在后台对公告、规则、信用和座位状态进行维护，这些结果会影响到

首页展示、预约预校验和推荐候选范围。系统架构的主要工作是将规则和数据统一起来。

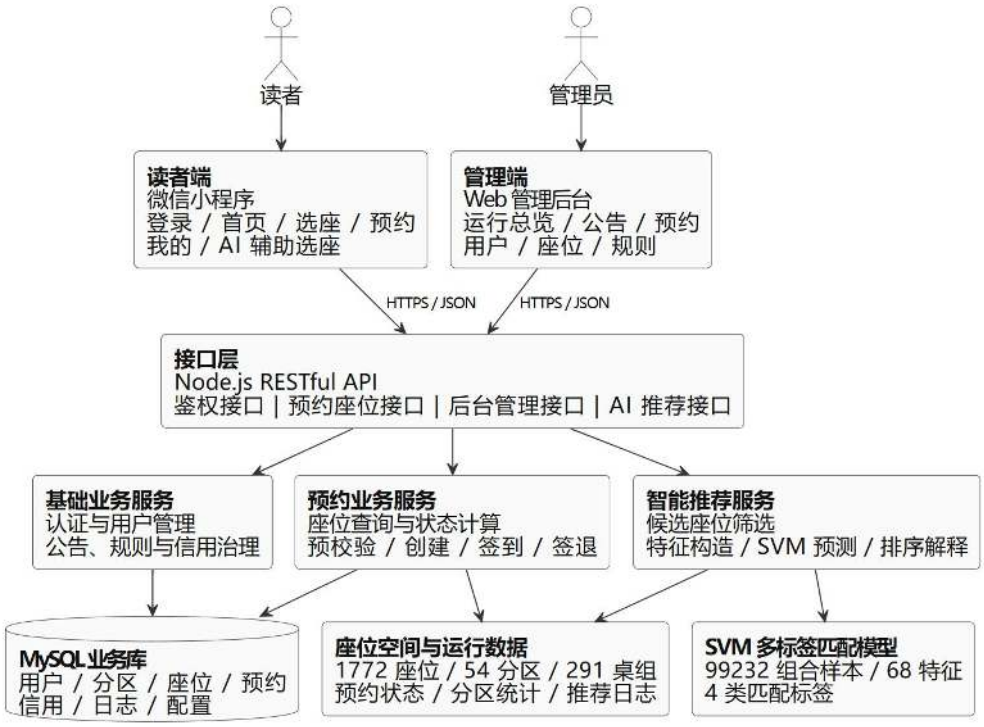


图 4.1 图书馆智能预约占座系统总体架构图

图 4.1 展示了系统整体的技术架构，微信小程序和 Web 后台构成了两种用户交互入口，利用 Node.js 业务接口可以调用用户信息、座位数据、预约记录、公告内容和使用规则等多种数据，MySQL 数据库负责存储静态资源和保存动态业务数据，算法模块根据用户需求文本、待匹配候选座位集合、68 项匹配特征进行座位适配预测和优先级排序，该架构包括读者预约操作、后台管理运维和 AI 智能推荐三种主要功能。

4.2 系统总体架构设计

系统功能结构按照用户角色和业务边界进行划分，主要包括读者端功能、管理员端功能和智能推荐功能三个部分。

读者端按照一次预约的使用顺序来组织功能，用户登录后先查看公告和开放分区，然后按照日期、时段、分区和环境偏好来筛选座位，系统会在提交之前进行预约预校验，对时间、座位状态、冲突预约进行检查，预约成功之后，用户可以在当前预约页面进行签到、暂离、返座、签退或者取消操作，并且能够通过个人中心查看身份、信用、历史记录。

管理员端的主要功能是进行资源运维和平台规则的管控，后台主页面可以直观地呈现场馆总座位数、累计预约量、空间使用率、各区域负载情况等主要运营数据，公告管理模块主要负责前台用户端的通知发布工作，预约管理模块用来处理各种预约异常记录，用户管理模块会对账号信用分值进行动态维护，分区和座位管理模块会实时更新对应资源的使用状态，规则配置模块则可以对预约有效时长、签到有效窗口期、允许暂离次数、违约信用扣分值等重要规则参数进行调整。后台操作的全部变更结果会同步作用在小程序端的公告展示内容上，并且会影响到座位的可预约状态、预约申请前置校验的结果。

图书馆智能推荐功能服务的对象主要是两类读者，一类是那些对场馆空间布局不太熟悉的使用者，另一类是有明确学习场景偏好的读者，用户在提交包括安静氛围、靠窗位置、供电配套和适配长时间学习等需求的检索文本之后，系统会先对用户提出的偏好要求进行语义分析，然后结合当前日期、具体时段、座位实时占用情况来筛选出初步候选座位集合，接着通过 SVM 多分类模型测算候选座位分别适配深度专注、舒适久学、设备作业和开放协作四类场景的概率，最后根据用户显式偏好匹配度、需求冲突项和场馆业务运行规则综合计算座位推荐优先级排序。

表 4-1 系统功能模块划分

模块类型	功能模块	主要设计内容
读者端	首页概览	展示公告、开放分区、座位总览和快捷入口
读者端	分区与选座	支持分区浏览、座位筛选、桌组查看和座位状态展示
读者端	预约管理	支持预约预校验、预约创建、预约取消和预约记录查看
读者端	履约反馈	支持签到、暂离、返座、签退和信用状态查看
管理端	运行总览	展示预约数量、座位使用情况和运营统计信息
管理端	业务治理	支持公告管理、预约处理、用户信用调整和规则配置
管理端	资源维护	支持分区信息、座位资源、座位状态和环境标签维护
推荐端	AI 选座	自然语言需求解析、候选座位筛选、模型预测和推荐排序

4.3 数据库设计

数据库设计是系统稳定运行的主要基础，本文系统拥有用户身份、图书馆空间资源、预约业务、签到签退、暂离记录、信用治理、后台管理、系统规则等多种数据，为了降低数据冗余并维持预约流程一致，本文在数据库设计中运用了将静态资源数据和动态业务数据分离开来的思路，数据库设计通常需要经过需求分析、概念结构设计、逻辑结构设计、物理结构设计、数据库实施、运行维护等步骤，这样可以使数据组织和业务流程保持一致[21]。

系统存储的静态资源数据主要分为场馆分区和座位两种基础信息，分区表包括楼层编号、区域唯一标识码、区域名称、运营启用状态、排序规则等重要字段，

座位表保存座位编号、所属分区、关联桌组编码、同组内排序号、座位类型、最大容纳人数、插座配备情况、是否临窗、安静度等级、环境属性标签、基础使用状态等信息，桌组没有被定义为独立的复杂实体，而是通过桌组编码和对应座位容量建立映射关系，主要用于前端页面展示、用户检索条件筛选、推荐算法特征统计三种场景。

动态业务数据主要包括预约主表、预约时隙表、签到签退记录表和暂离记录表，预约主表记录了用户信息、座位信息、日期信息、开始时间和结束时间，并且对预约状态进行记录，时隙表将预约时间划分成固定粒度的占用记录，从而对同座位同时间段的冲突进行限制，签到签退表和暂离表记录预约之后的履约过程，用来给当前预约页状态的回显和后台异常处理提供依据。

数据治理和配置模块主要为后台管理功能提供支持，用户信用日志专门记录信用分加减的触发原因，公告表用来存储首页推送的各类通知内容，管理员操作日志对后台执行的所有主要操作进行记录，座位状态日志完整保存座位维护、停用及恢复的整个流程操作记录，系统配置表则存储预约时长上限、签到有效窗口期、用户暂离允许次数、黑名单触发阈值、违约行为扣分量等重要运行参数。这些全部数据可以给平台规则的动态调整、异常运营事件的处理提供决策参考，也可以满足各种操作和事件的回溯核查需要。

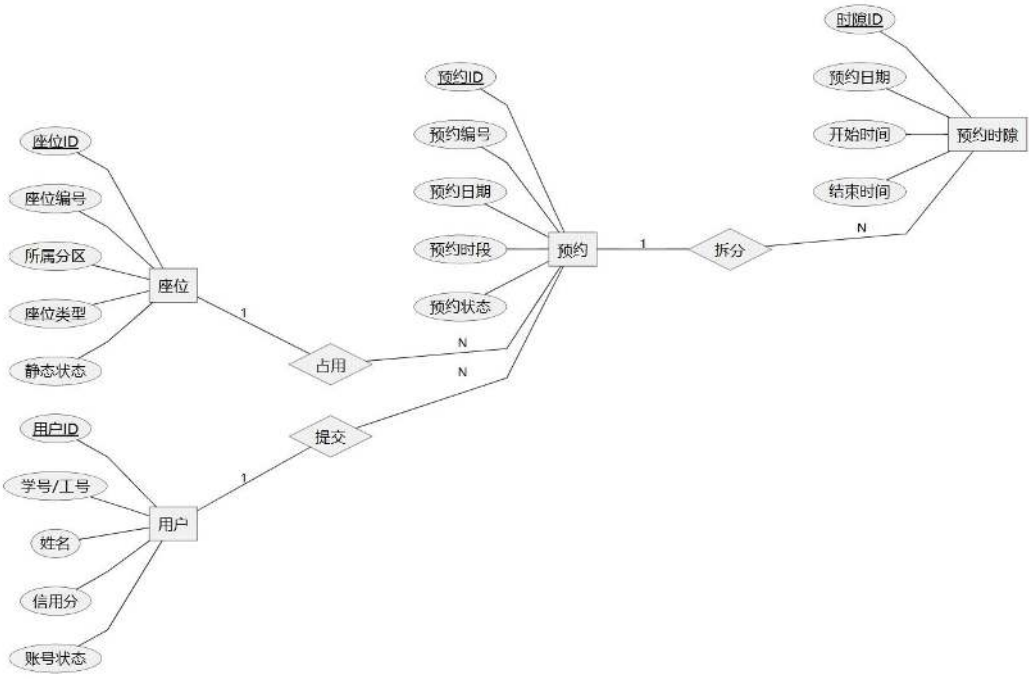


图 4.2 核心业务数据库 E-R 关系图

图 4.2 说明预约业务中主要实体的关系，用户可以生成多条预约记录，每条预约绑定一个座位和一个日期时间段，同一个座位在不同时间段内也会对应多条预约，系统把一次预约拆分成多个时隙记录，以防止仅根据起止时间字段来判断

冲突，同时利用预约与时隙的一对多关系控制占用，该系统为预约预校验、冲突判断、签到签退和后台释放预约提供了数据基础。

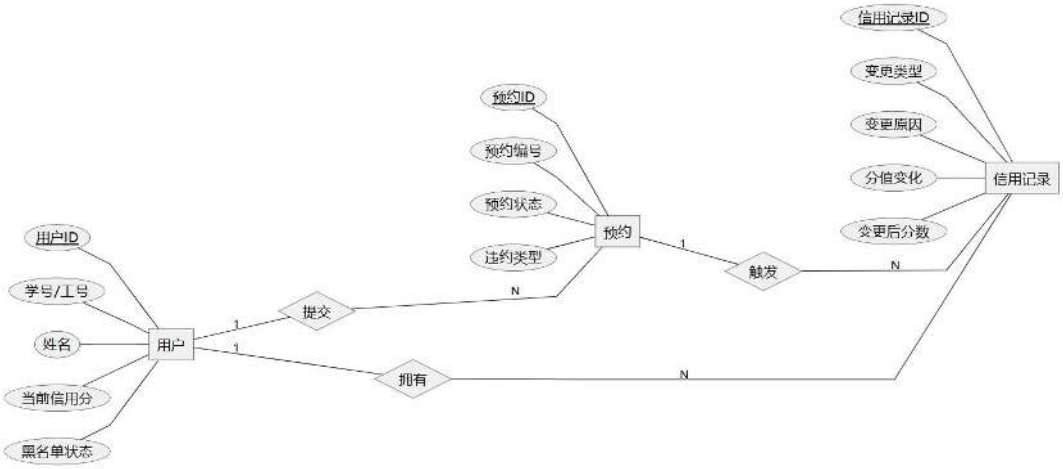


图 4.3 用户信用实体属性 E-R 图

图 4.3 展示了用户信用治理相关实体之间的关系。用户在系统中可以提交多条预约记录，同时也会拥有多条信用记录；当预约出现未签到、超时暂离、人工处罚等异常情况时，预约记录会触发相应的信用变更，并在信用记录中保存变更类型、变更原因、分值变化和变更后分数等信息。这样的设计使信用分不是简单保存在用户表中的静态字段，而是能够通过信用记录追踪每一次分值变化的来源，管理员在后台进行核查时，也可以根据预约编号和信用记录还原违约处理过程，从而增强系统规则执行的透明度和可追溯性。

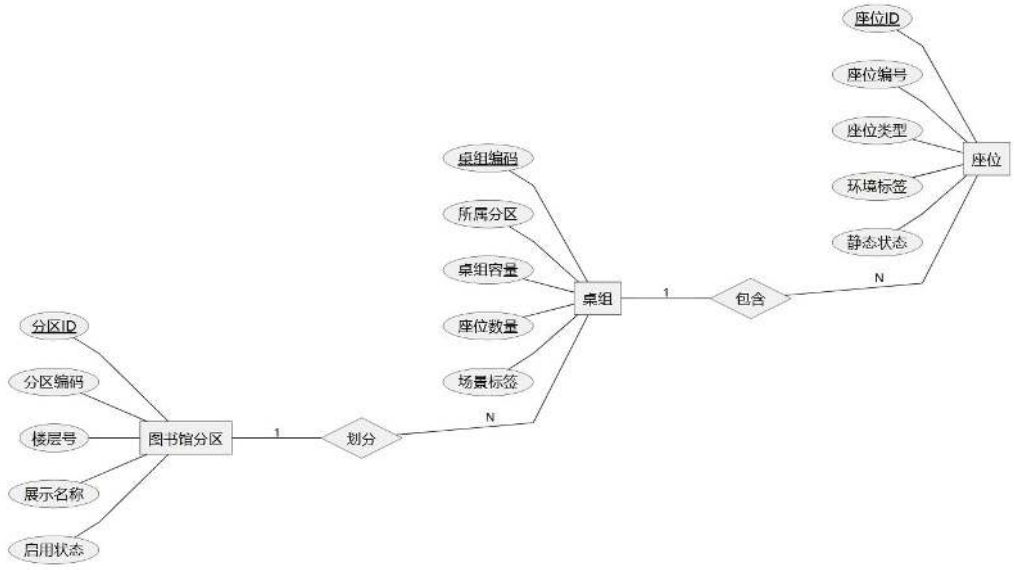


图 4.4 空间资源实体属性 E-R 图

图 4.4 展示了图书馆空间资源的分层组织方式，系统把分区当作上层单元，分区下面按照桌组编码来组织座位，座位则保存编号、容量、插座、靠窗、安静

程度和状态等属性，在进行推荐数据处理时，桌组编码可以用来对插座率、靠窗率、安静率、通道干扰这些特征进行统计，该种组织形式让平面图展示、分区筛选、具体座位预约、推荐模型输入均可以利用同一套空间编号。

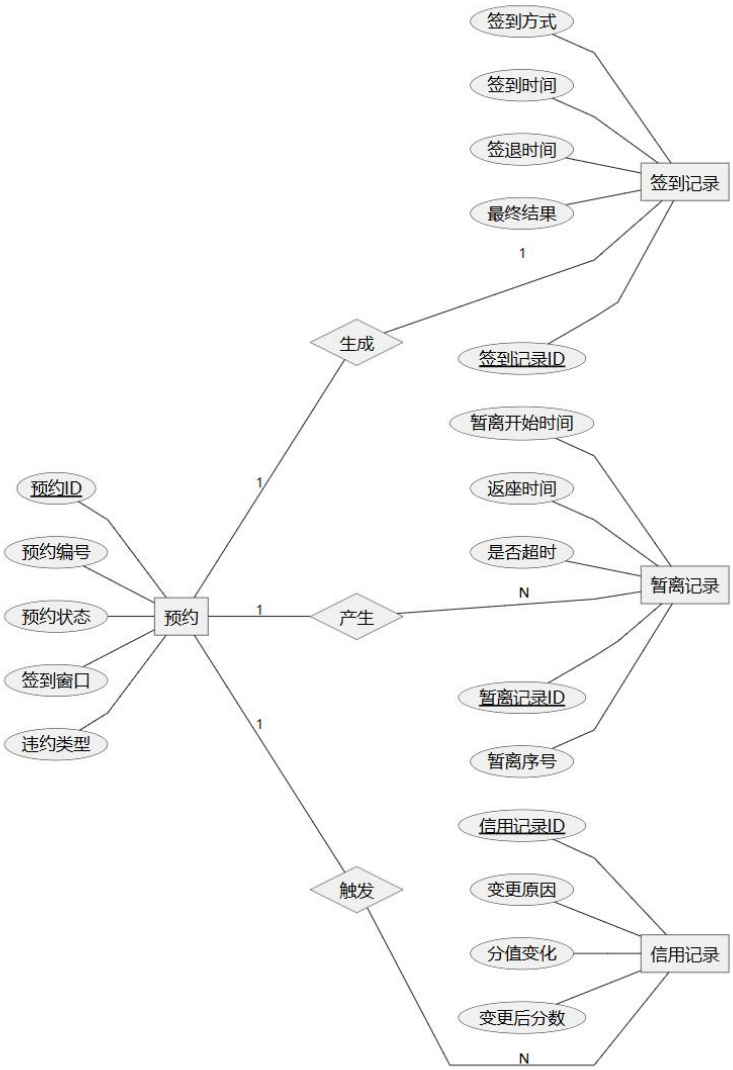


图 4.5 预约履约实体属性 E-R 图

图 4.5 表明预约成功之后履约状态的记录关系，预约实体含有预约编号、预约状态、签到窗口和违约类型等信息，是整个履约过程的中心，用户到达图书馆后，系统利用预约来生成签到记录，该记录保存了签到方式、签到时间、签退时间和最终结果等数据，用户暂时离开座位时，系统会生成暂离记录，用来记录暂离开始时间、返座时间和是否超时等信息。如果在预约过程中出现未签到、暂离超时或者其他违规情况，相应的预约也会使信用记录发生改变，该设计使预约业务不限于创建预约的操作，也能够持续追踪签到、暂离、返座、签退和违约处理等后续状态，为读者端状态显示和后台异常处理提供依据。

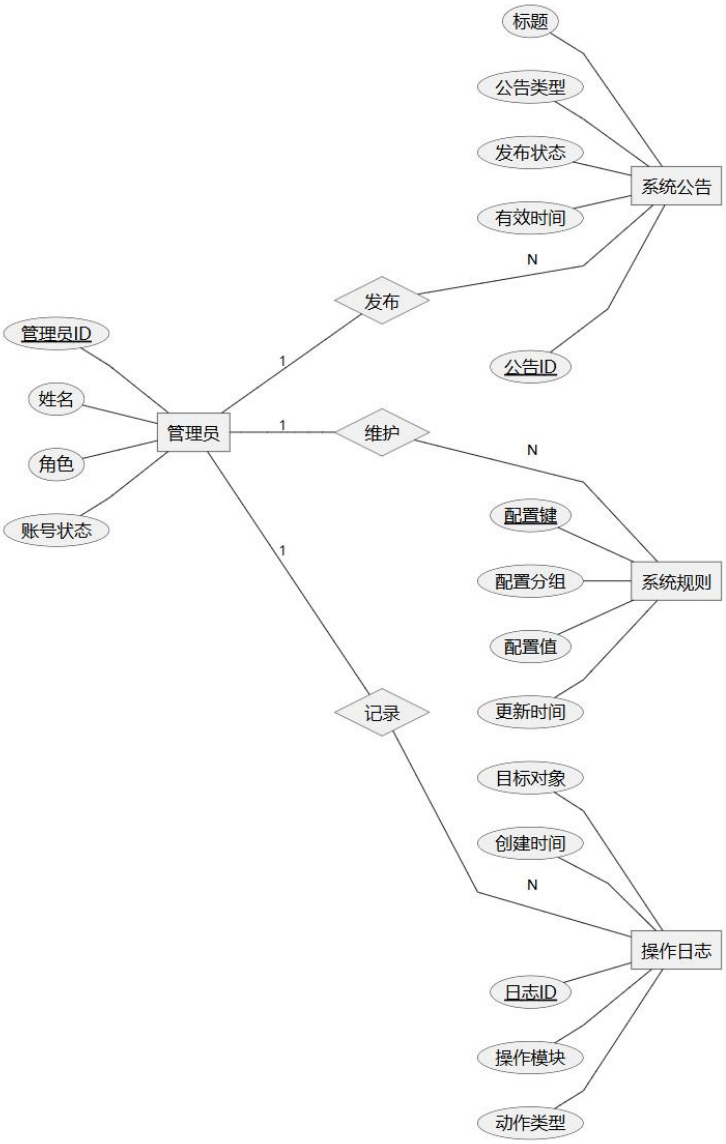


图 4.6 后台治理实体属性 E-R 图

图 4.6 展示了后台治理模块的数据关联逻辑，管理员可以对发布公告进行批量操作，单条公告记录包括标题、类型、发布状态和有效期限这四个主要字段，公告在发布之后会自动同步到小程序前端首页进行展示，管理员可以对系统运行规则进行动态维护，规则表利用配置键、配置分组、参数值这三个方面来存储预约、签到、临时离场、信用管理四类业务的主要参数，全部重要操作都会被写入操作日志，日志具体记录操作所属模块、动作类别、操作目标对象及触发时间，后台管理系统可以通过上述多张数据表来完成相关数据的查询调取，也可以以此形成全链路可追溯的治理留痕。

4.4 预约业务流程设计

预约业务流程是本文系统的核心流程。为了避免出现重复预约、无效预约和状态不可追踪等问题，系统采用“先校验、再提交、后履约”的流程设计。

用户选定预约日期、时段和座位之后，系统会对预约进行预先校验，校验内容包括核验用户身份的有效性、排查黑名单状态、判断预约日期和时段是否合规、检查座位是否可用、确认对应时段的座位是否存在其他预约，在这些校验全部通过后，前端会赋予用户正式提交预约的权限。

预约创建完成后，系统会把预约主记录写入数据库，并且依据预约的时间范围来生成对应的时隙占用记录，预约开始前会开放签到入口，用户到达图书馆后进行签到，在使用期间，用户可以按照规定暂时离开并返回座位，预约结束时进行签退，当出现未签到或暂离超时等状况时，后端系统会对预约状态进行更新，同时释放相关时隙，并且把信用变化记录在日志中。

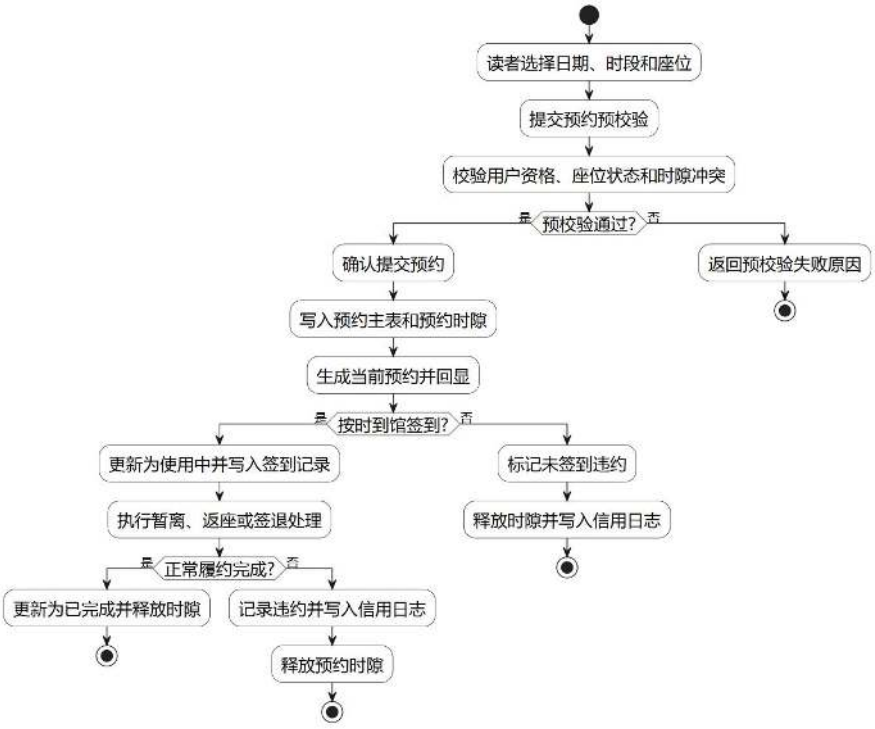


图 4.7 预约业务流程图

图 4.7 说明了读者选座并完成预约的过程，读者在小程序端选定日期、时段和座位后，后端会对用户资格、座位静态状态、预约时间、时隙占用情况进行校验，系统将预约主表和时隙表进行后写入，并且在当前预约页上把结果进行回显，预约生效以后，签到成功会使状态推进到使用中，暂离和返座会继续更新履约记录，正常签退则标记为已完成。如果发生违约行为，系统会记录违约的具体类型，并且释放相应的时隙，同时更新信用日志。

4.5 智能推荐流程设计

智能推荐流程的主要目标是从可预约座位池里选出和用户当前需求匹配程度更高的候选对象，本文没有用把座位分到固定类别的方式，而是把用户需求和

候选座位组合成一个匹配样本，主要原因在于同一个座位对不同需求的适用价值有明显区别，而且单个用户需求通常同时包括安静度、充电方便性、采光情况、桌型大小等各方面要求，根据这个特点，本文把推荐流程分成需求解析、候选集过滤、特征模式形成、SVM 多标签预测和推荐结果排序这五个主要部分。

需求解析环节的主要工作是提取用户明确提出的各种偏好要求，具体包括喜欢安静环境、需要靠窗的位置、座位配有充电插座、可以支持长时间学习、能够满足多人讨论、距离人员通行通道较远、靠近藏书书架等，候选过滤阶段会对预约日期和具体时段进行关联校验，去除已经被其他人预约、暂时关闭或者不符合场馆使用管理规定的座位，特征建立阶段把当前用户的需求条目和各个候选座位的属性项一一对应拼接，形成和模型训练阶段完全相同的匹配特征，从而保证在线预测和离线训练的数据口径保持一致。

模型预测阶段调用训练后的 SVM，得到深度专注、舒适久学、设备作业和开放协作四类概率；排序阶段则将模型概率、显式偏好命中、冲突项和座位可用状态合并为综合匹配度，返回 Top-N 推荐座位。推荐理由用于说明座位满足了哪些需求条件，或者存在哪些可能降低得分的因素，使用户能够理解系统给出该候选座位的依据。

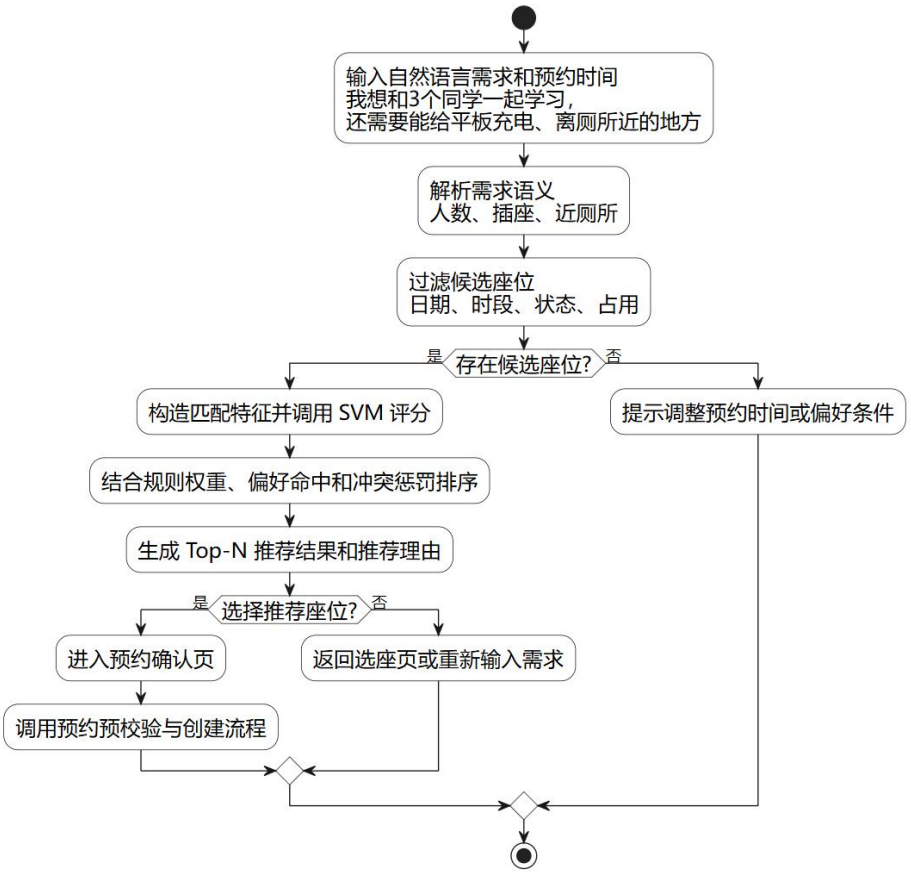


图 4.8 智能推荐流程图

图 4.8 显示了 AI 选座的全部过程，用户在提交座位需求、确定使用日期和具体时段之后，系统会解析用户提出的插座可用性、靠窗位置、环境安静度、采光条件、多人协同学习、远离过道等偏好，并且结合实时座位状态、各时段占用数据、场馆预约规则来完成候选座位的初筛，经由业务规则过滤所得座位集合同用户当前需求共同组成匹配样本。系统在提取 68 项特征并建立模型后，将座位输入 SVM 模型，输出四类标签的对应概率，后端系统会综合考虑用户的偏好情况、命中记录、冲突项，来计算各个候选座位的综合得分，然后按照排序结果返回 Top-N 个推荐座位。

4.6 本章小结

本章主要说明系统的整体设计方案，中心整理读者端、管理后台、后端接口、数据库、推荐模块五个部分的联系逻辑：在系统架构上，微信小程序是读者预约和 AI 选座的前端入口，Web 管理后台进行公告发布、规则设置、座位调度和信用管理等运维工作，Node.js 接口层统一进行业务逻辑校验和数据交互调度，MySQL 数据库保存静态基础资源和动态业务运行数据，推荐模块先根据规则筛选出候选座位池，然后进行特征提取建立和 SVM 模型预测。数据库设计环节按照静态基础资源、动态预约数据、管理配置参数三类数据方面，梳理出用户、空间分区、座位、预约记录、签到、暂离、信用、公告、规则等主要实体的关系结构，流程设计部分分别确定了预约履约全流程与智能推荐的实现方式，本系统的整体架构可以实现前端预约操作、后台运维管理、用户需求和座位资源匹配推荐三者之间的高效数据共同工作。

第五章　系统实现与算法设计

5.1 小程序端实现

小程序端按读者操作顺序拆分页面模块。首页通过 `/api/home` 获取公告、系统摘要、热门分区和快捷入口，分区与选座页面根据日期和时间窗调用 `/api/zones`、`/api/seats`，预约确认页先请求 `/api/reservations/preview` 完成预校验，校验通过后再提交创建预约。通过这种实现方式，页面展示和预约规则都能够依赖后端实时数据。

座位浏览功能采用平面图分区、分区内桌组、桌组下属座位列表所组成的三级层级模式，系统未将座位和平面图片素坐标进行硬性绑定，而是利用分区编码、桌组编号和座位序号的关联方式，实现平面图界面和后台数据库记录的映射，用户能够先对空间区域分布形成整体认识，随后逐层进入对应桌组以查看具体座位信息，座位编号、实时预约状态、筛选匹配结果、个性化推荐内容均可以从同一套结构化数据集中调取。

小程序端也应当对用户状态进行及时反馈，预约页面目前展示预约状态、取消条件、签到签退入口和异常提示，个人中心集中显示身份信息、信用分、预约统计和快捷入口，读者可以使用这些页面来确认自己目前是否拥有预约资格，并且查看是否需要进行签到，同时也可以检查信用状况是否会继续影响后续的操作。

5.1.1 首页总览与场景发现

小程序端页面围绕预约流程组织，主要包括首页、空间导览、选座页、预约确认页、当前预约页、AI 选座页和个人中心页。页面通过统一封装的请求方法与后端通信：首页读取 `/api/home`，选座页根据日期、时段和偏好读取 `/api/zones` 与 `/api/seats`，预约确认页调用 `/api/reservations/preview`，当前预约页则通过签到、暂离、返座和签退接口推动状态变化。

系统交互流程将首页入口视为起点，依次包括地图筛选、分区详情查看、座位预约、预约队列查询这五个主要环节，首页和选座页面都设有 AI 辅助选座功能入口，页面显示的分区编码、桌组属性、环境特征标签和座位可预约状态均依靠后端接口获取，前端代码并未进行硬编码，该设计确保地图浏览环节具有直观易用性，并且让预约校验规则、AI 推荐结果随数据库实时状态同步更新。



（a）首页首屏概览



（b）偏好与热点分区



（c）常用场景入口



（d）常用场景详情

图 5.1 首页总览与场景发现界面

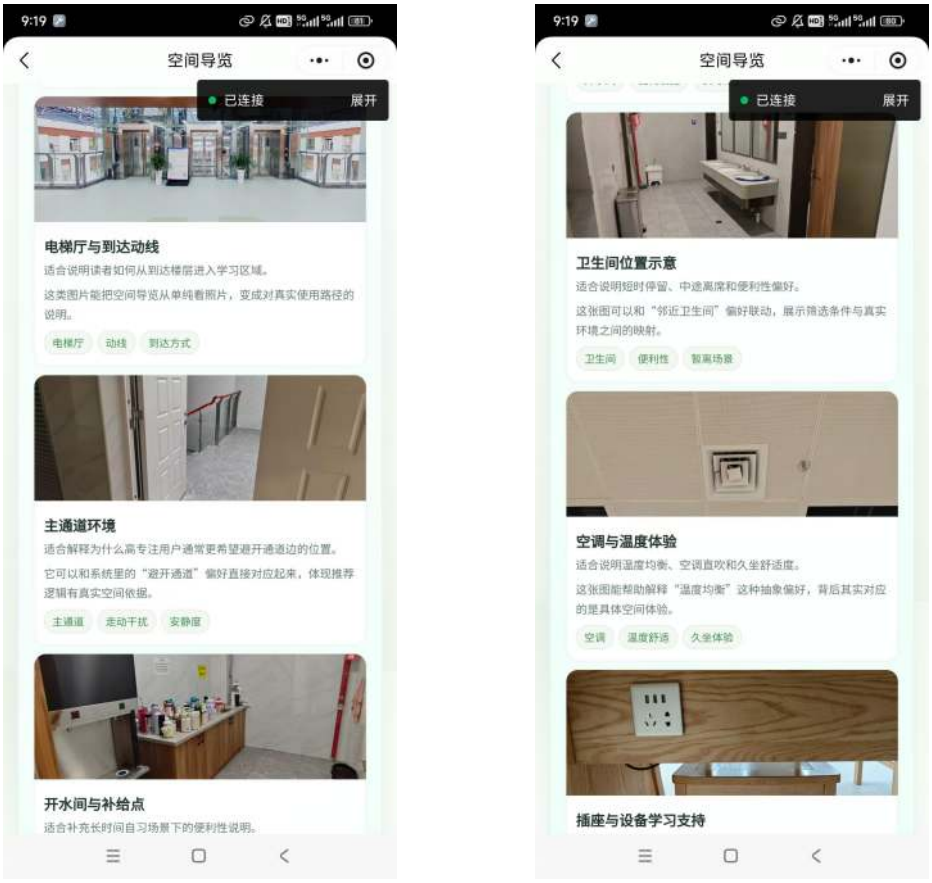
图 5.1 展示了首页总览与场景发现模块的运行效果。图 5.1(a) 中，首页首屏通过馆舍实景背景、用户问候、系统名称和资源指标展示当前系统概况，用户可以直接看到开放分区、总座位和预约时长等关键信息；图 5.1(b) 展示了首页下方的快捷入口和热点分区，用户可从“座位预约”“AI 选座”“空间图集”等入口进入不同业务流程，也可以点击热门分区直接跳转到对应区域的选座页面；图 5.1(c) 和图 5.1(d) 展示了常用场景入口及其详情内容，系统将“冲刺学习、临窗阅读、结伴自习”等学习目标与插座、靠窗、安静、桌型等座位属性进行关联，使用户不必先理解全部字段含义，也能通过场景化方式进入选座和偏好筛选流程。

从代码实现上看，首页页面主要由 `pages/home/index.js` 和 `pages/home/index.wxml` 共同完成。`index.js` 中的 `loadPageData()` 负责请求首页数据，并将接口返回的公告、热门分区和资源摘要转换为页面可直接渲染的数据结构；`buildHotZoneCards()` 用于生成热门分区卡片，`buildNoticeCards()` 用于整理公告展示内容；`index.wxml` 则根据这些数据渲染首屏指标、快捷入口、公告提醒、偏好速选和热门分区列表。用户点击“座位地图”时，页面会跳转到选座页；点击“AI 选座”时，会进入智能选座入口；点击热门分区时，则携带分区编码跳转到对应分区的座位浏览页面。通过这种实现方式，首页既承担信息总览作用，也成为连接分区浏览、环境偏好、预约流程和 AI 推荐功能的导航中心。

5.1.2 空间导览与环境特征展示

座位环境会直接影响到读者的选座体验，比如靠窗与否、插座存在与否、是否靠近主通道或卫生间等因素会影响到学习感受，为了让读者在预约前先理解这些空间差异，小程序增加了一个空间导览模块，通过照片和简短说明来展示馆内常见位置和服务设施，并且给后续地图筛选和 AI 推荐中的环境标签提供解释基础。

用户从首页入口进入空间导览，页面内容来源于图集配置和环境特征说明，系统把电梯口、主通道、开水间、插座位、靠窗位等空间要素整理成图文项，该模块没有直接创建预约，但是其中的插座、靠窗、安静、采光、远离通道等口径与座位筛选和推荐特征保持一致，读者理解这些标签后，在正式选座时更容易表达自己的偏好。



（a）位置与服务设施导览　　（b）环境特征与设备设施导览

图 5.2 空间导览页面示意

图 5.2 表明小程序空间导览页面的运行情况，图 5.2(a)通过图文卡片的形式，对电梯厅与到达动线、主通道环境、开水间与补给点等位置及服务设施进行展示，让读者在预约之前能够了解到馆内的动线、常用设施的分布情况，图 5.2(b)继续展示卫生间位置、空调与温度体验、插座与设备学习支持等环境特征，读者可以将后续选座筛选和推荐理由中的主通道、便利性、温度舒适、设备学习等标签对应到实际空间。该模块没有直接进行预约，但是可以给地图筛选和 AI 辅助选座提供比较直观的空间说明。

5.1.3 地图筛选与分区详情

选座流程先要求用户确定日期和时段，再通过地图缩小分区范围，最后进入分区详情查看桌组和具体座位。与一开始展示全部 1772 个座位相比，这种分层浏览方式能够降低信息量，也更接近读者实际到馆时先判断区域、再选择座位的习惯。

地图筛选相关操作主要在选座页面进行，页面完成初始化后，会把当前选定的日期、预约时段的起止时间、用户预先设置的偏好参数一起提交给后端，这样

可以得到各区域的分区编码、可预约座位余量、区域拥挤度、配套环境统计数据，当用户点击任意分区时，前端会对该分区对应的座位列表发起查询请求，返回的座位数据会按照桌组编码归类后显示。因为平面图的热点标注区域和后端存储的分区编码有唯一对应关系，分区的实时状态、系统推荐的最优分区排序都可以根据接口返回的最新数据进行动态调整。



（a）选座工作台　　（b）地图筛选界面　　（c）分区详情与推荐座位

图 5.3 地图筛选与分区详情界面

图 5.3 展示了选座工作台、地图筛选和分区详情的页面效果，用户首先在图 5.3(a)中选取日期、时段和入口，图 5.3(b)按照筛选条件使用不同颜色来标记分区资源的状态，图 5.3(c)显示具体分区内的座位推荐和环境预览，图 5.3(d)解释 Q3-A-01 这类座位编码的生成方式，该编号被前端展示、数据库记录和推荐模型输入所使用。

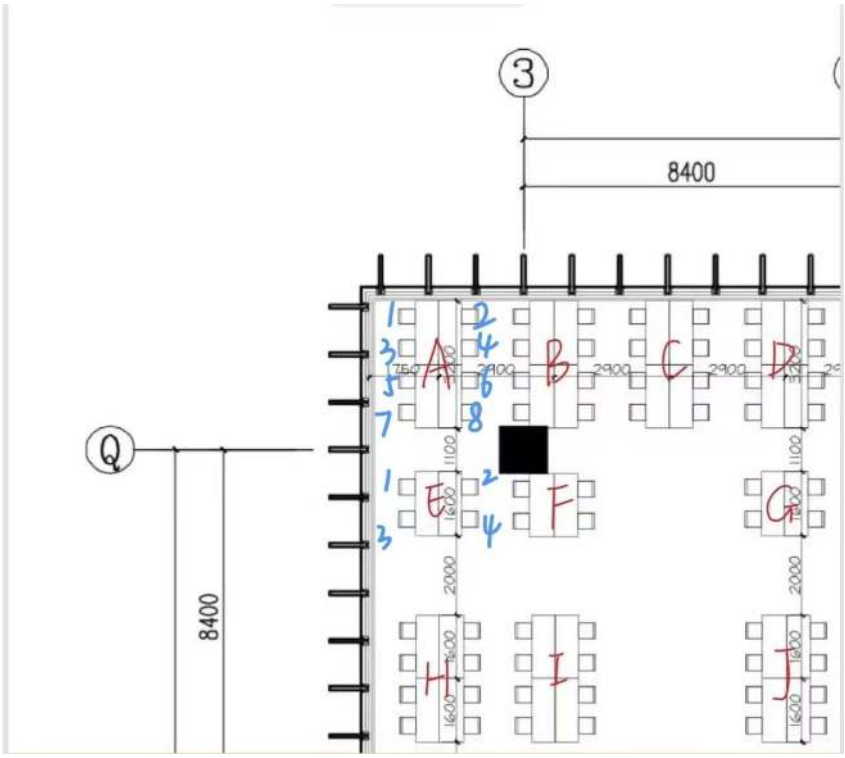


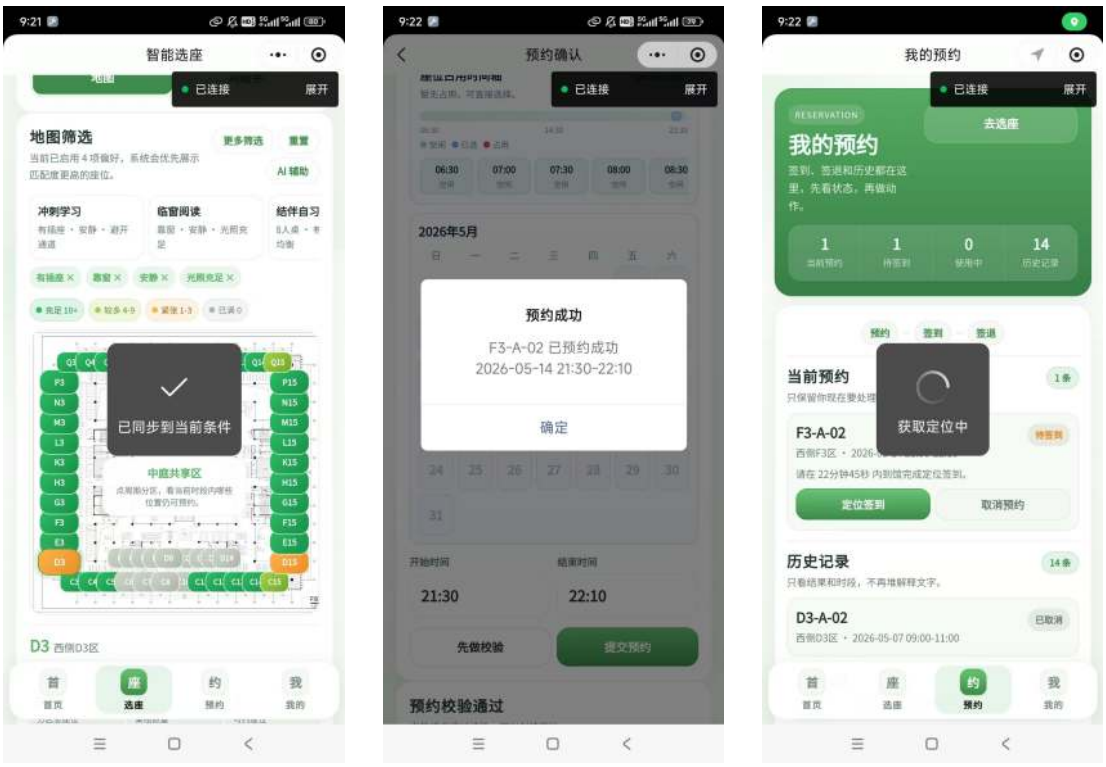
图 5.3(d) 平面图分区与桌组编号标注示意

图 5.3(d) 表明了局部平面图中桌组的标注方法，座位编码首先取分区编号，然后取桌组字母，最后加上座位序号，比如 Q3-A-01，该编码使页面中的座位点位对应到座位表中的具体记录。

5.1.4 预约创建与履约状态流转

预约建立、履约状态流转组成了读者端功能链路的主要部分，用户在分区浏览、座位筛选和时间选择之后，会先进行预校验，然后再正式提交预约，预约成功以后，当前预约页面仍然能够提供签到、暂离、返座、签退、历史记录更新这些功能，本文选取地图筛选同步、预约成功、定位签到、开始暂离、返座成功和签退成功六个界面，用来展示状态从待签到到使用中、暂离中和已完成的变化过程。

用户在选座界面点击确认预约按钮后，前端会把座位编号、区域名称、预约日期、时段起止信息当作参数进行传输，在正式提交之前先请求/api/reservations/preview 接口，依次核验用户预约权限、预约时段合规性、座位当前使用状态、同一账号的预约冲突和目标座位的占用冲突，这些校验全部通过之后再调用/api/reservations 接口以完成预约订单的创建。后续预约页面会依据后端返回的实时状态，分别显示待签到、使用中、暂离、已完成四种状态，并且可以调用签到、暂离、返座、签退等接口来完成全流程操作。



（a）地图筛选条件同步　（b）预约创建成功　（c）定位签到处理

（d）开始暂离状态　（e）返座成功反馈　（f）签退成功反馈

图 5.4 预约创建与履约状态流转界面

图 5.4 展示了读者端从选座条件同步到预约创建、签到、暂离、返座和签退的主要状态变化。图 5.4(a) 中，用户在地图筛选页选择插座、靠窗、安静和采光

等偏好后，系统会将条件同步到当前选座流程，并在平面图中展示不同分区的可预约情况；图 5.4(b) 表明预约预校验通过并提交后，系统能够返回预约成功提示，同时给出座位编号和预约时间段；图 5.4(c) 展示了定位签到成功后的界面状态，当前预约由待签到进入使用中状态，页面同步提供暂离和直接签退等后续操作入口；图 5.4(d) 展示了用户暂离后的状态反馈，当前预约进入暂离中状态，并提示用户需要在规定时间内返座；图 5.4(e) 展示了返座成功后的界面状态，预约继续保持使用中，用户仍可继续使用座位或执行签退操作；图 5.4(f) 展示了签退成功后的结果，当前预约数量清零，记录进入历史列表。通过这一组界面可以看出，小程序端已经把选座、预约、签到、暂离、返座和签退连接成连续的履约流程。

5.1.5 AI 辅助选座界面

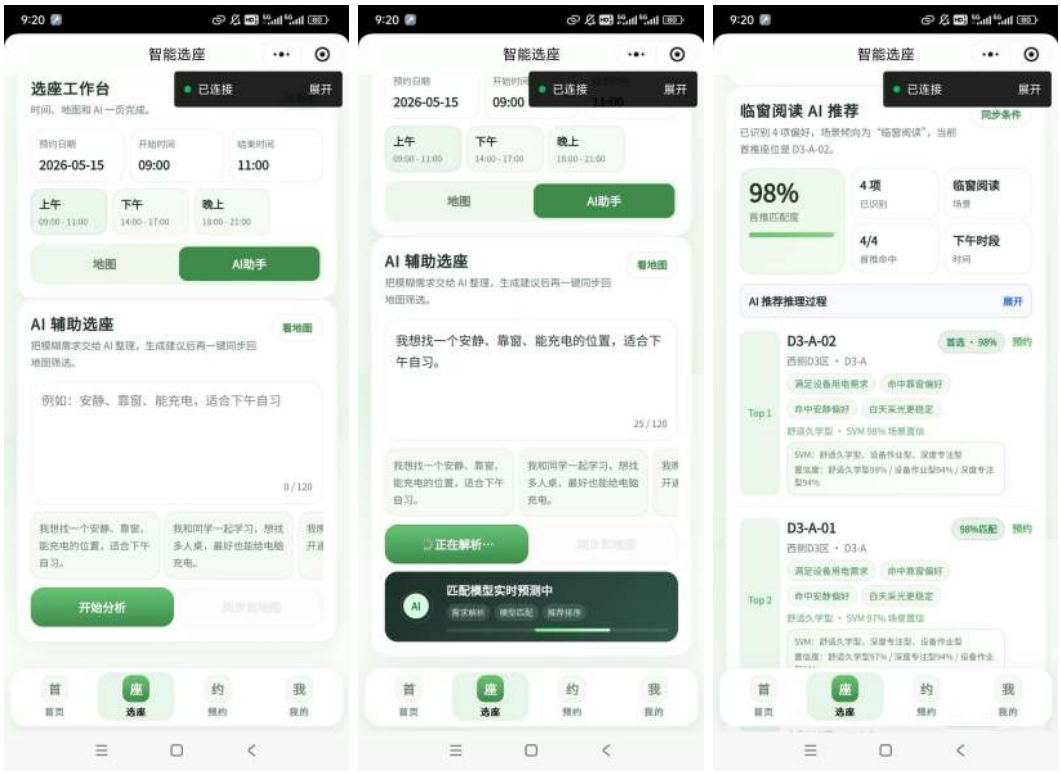
除地图筛选外，本文还设计了 AI 辅助选座入口。传统地图筛选更适合已经明确分区、座位属性和预约时间的用户，例如用户已经知道想去哪个楼层、哪个区域，或者只需要按照插座、靠窗、安静等条件进行简单过滤。但在实际使用中，部分读者并不一定熟悉图书馆内部空间分布，也不一定能够准确判断某个分区是否适合自己的学习任务。他们更常表达的是“想找一个安静、能充电、适合长时间学习的位置”或“希望找一个适合多人讨论但不要太吵的位置”这类自然语言需求。如果仍然完全依赖地图筛选，用户需要反复切换分区、查看座位属性和比较候选结果，操作成本较高。因此，AI 辅助选座功能的设计目标，是在不改变原有预约规则的前提下，为用户提供一种从自然语言需求直接进入候选座位推荐的辅助入口。

AI 辅助选座功能的实现需要依靠需求录入界面、推荐结果呈现界面和常规选座流程的共同运作，用户在完成座位需求填报并确认预约日期和时段之后，前端会自动调用/api/assistant/seat 接口，把用户提交的需求文本和对应的时间参数传输到后端，后端没有直接对全部座位进行排序计算，而是先执行与常规预约流程一致的过滤校验步骤，校验内容包括座位是否处于启用状态、是否正处于维护期、对应时段是否已经被预约、预约时间是否符合平台管理规则。座位只有通过全部过滤校验后，才会进入后续的模型预测流程。

在候选座位确定后，后端会对用户需求进行偏好解析，将自然语言中的安静、靠窗、插座、采光、长时间学习、多人讨论、远离通道、靠近书架等描述转化为结构化偏好特征。随后，系统会将同一条用户需求与每一个候选座位逐一组合，结合座位环境属性、桌组信息、分区统计、偏好命中项和冲突项构造运行时需求-座位匹配特征。这些特征与模型训练阶段使用的特征保持一致，能够保证离线训练模型和在线推荐调用之间的数据口径一致。完成特征构造后，后端调用训练

后的 SVM 多标签匹配模型，对候选座位进行批量预测，输出深度专注匹配、舒适久学匹配、设备作业匹配和开放协作匹配四类标签概率。

推荐结果返回到小程序端后，页面不仅展示候选座位编号，还展示推荐排名、综合匹配度、命中标签和推荐理由。推荐理由用于说明候选座位为什么适合当前用户需求，例如座位是否靠窗、是否具备插座、是否处于较安静区域、是否适合长时间学习，或者是否存在靠近通道、桌型不匹配等可能降低推荐分数的因素。用户可以根据推荐理由判断是否接受系统推荐，也可以返回地图筛选页继续手动调整条件。也就是说，AI 辅助选座并不是替代用户选择，而是在用户表达需求后提供一组更符合学习场景的候选结果，并把推荐结果继续接入原有的座位确认和预约提交流程。



（a）AI 需求输入界面　　（b）SVM 实时预测状态　　（c）AI 推荐结果展示

图 5.5 AI 辅助选座界面

图 5.5 展示了 AI 辅助选座功能的运行界面。图 5.5(a) 中，用户可以直接输入“安静、靠窗、充电、多人学习”等自然语言需求；图 5.5(b) 展示了系统调用推荐接口时的等待状态，说明该过程需要后端完成候选座位筛选和模型预测；图 5.5(c) 展示了推荐结果页，页面不仅给出候选座位，还展示部署模型、候选座位数量、Macro-F1、Micro-F1、匹配标签、匹配度和推荐理由等信息。该设计使用户能够看到推荐依据，也说明第五章中的模型接入已经落实到小程序实际交互流程中。

5.1.6 个人中心与规则反馈

个人中心模块的主要功能是向读者展示自身的预约状态和信用情况，页面显示的内容包括用户基础信息、信用评分、累计预约总量、已履约完成次数、违约记录次数、黑名单判定阈值及历史信用明细，系统在运行加载阶段会调用对应接口来获取用户个人档案和信用行为日志，然后对返回的数据进行结构化处理，分别以信用信息卡片、统计指标模块、使用规则说明这三种形式进行展示，因为信用评分是预约资格预校验环节中的主要判定依据，个人中心同时也承担着解释信用规则和提示预约风险的功能。



图 5.6 个人中心与信用反馈界面

图 5.6 展示了个人中心页面。用户可以直接看到当前信用分、黑名单阈值、扣分记录和规则说明，并且这些内容与后台规则配置和信用调整结果保持一致。因此，个人中心不只是个人资料页，也承担着连接读者端操作结果和后台治理规则的作用。

5.2 后端业务实现

后端接口层按照业务职责划分为认证、首页、规则、分区与座位、预约、个人中心、算法概览以及后台管理等多个接口组。其中，读者侧核心接口包括 /api/auth/register、/api/auth/login、/api/home、/api/rules、/api/zones、/api/seats、/api/reservations/preview、/api/reservations、/api/reservations/me 和 /api/profile/me 等；这些接口共同构成了小程序前端的主要交互基础。

后端预约逻辑主要依靠规则校验和状态同步来实现，在进行预约创建之前，接口会对用户身份、时间窗、座位状态和冲突关系进行检查，系统在创建后会进行对当前预约的维护，同时也会进行对取消预约的操作，并且会对后续状态的变化进行记录，后台接口可以执行代签退操作，也可以释放预约，并且能够发布公告和配置规则，建议接入/api/assistant/seat 和/api/algorithm/overview 这两个接口，其中/api/assistant/seat 接口用来实时生成座位推荐，而/api/algorithm/overview 接口负责展示算法实验和部署模型的信息。

在数据访问上，后端利用仓储层获取用户、座位、预约、公告、规则和后台管理数据，并且保存座位主数据和推荐运行数据的 JSON 文件，用来进行算法建立和演示联调，该组织形式把业务数据、空间数据和模型数据分别放在不同层进行处理，使开发调试、答辩演示和后续部署时能更容易发现错误。

5.3 管理后台实现

管理后台已经实现运营报表、公告管理、预约管理、用户管理、分区管理、座位管理和规则配置等视图。管理员登录后，可以查看总座位数、今日预约量、当前有效预约、信用预警和最近动态，也可以按照筛选条件查询预约记录，并执行释放预约、代签退等处理操作。

分区和座位管理可以按照区域编码、状态、关键字进行筛选，管理人员也可以对座位属性进行编辑，对分区进行启停操作，对维护状态进行调整，规则配置主要对预约时长、签到窗口、暂离时长、信用惩罚参数进行维护，管理员通过运用这些后台功能，在不改动小程序代码的前提下，能够处理发布公告、干预预约、治理信用、维护座位、调整业务规则等工作。

后台各类操作会对读者端的各项业务功能产生直接作用，公告发布后会直接展示在小程序首页，规则配置是通过调用/api/rules 和/api/reservations/preview 两个接口来对预约校验逻辑产生影响，用户信用额度调整结果会同步展示在“我的”页面，分区规则和座位信息的维护操作会直接调整选座页、分区详情页及 AI 推荐页的可选资源范围。可见后台系统不但是主要管理工具，而且也是保障前台业务稳定运行的主要支撑。

5.3.1 首页概览与运行态监测

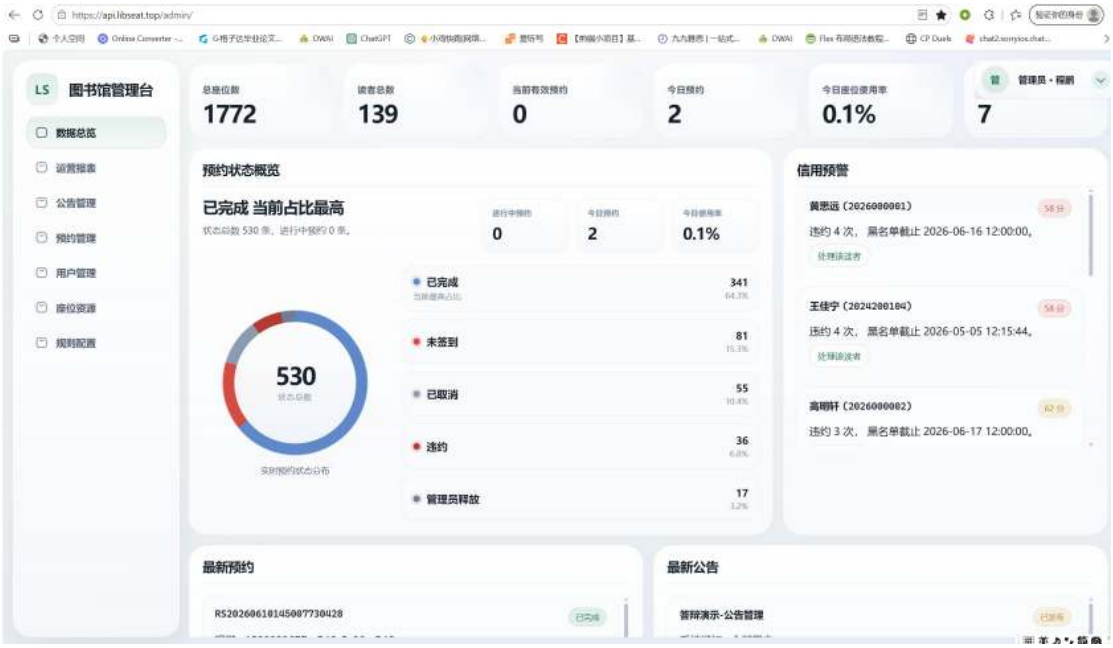


图 5.7 管理后台首页与运行态总览界面

管理后台首页的布局情况可以参考图 5.7，页面顶部依次展示场馆总座位数量、注册读者总人数、当前生效预约数量、当日预约总数量、当日座位使用比例和已发布公告数量，中部列出不同预约状态的比例分布，右侧列出有信用异常风险的用户名单，页面底部显示最新的预约记录和公告内容，管理员登录后台后，可以通过上述主要指标快速了解座位资源的整体利用情况、是否有异常预约行为及潜在信用风险，并据此决定是否需要发布运营公告、调整预约使用规则或人工干预释放被占用的预约名额。

从实现方式看，后台首页主要由 admin-web/app.js 中的 loadDashboard() 请求 /api/admin/dashboard 获取运行态数据，并由 renderDashboard() 渲染到页面。后端在 server/src/index.js 中接收该接口请求，再调用 admin-repository.js 中的 getAdminDashboard() 统计座位总数、读者总数、当前有效预约、今日预约、公告数量、预约状态分布、最新预约和信用预警等信息。前端根据 summary 生成顶部指标卡片，根据 statusDistribution 生成预约状态分布图，并将 recentReservations、recentNotices 和 creditWarnings 渲染为最新预约、最新公告和信用预警列表。通过这种方式，后台首页能够把分散在座位、预约、公告和用户信用中的运行数据聚合为管理员可直接判断的运行态视图。

5.3.2 运营报表与分区负载分析

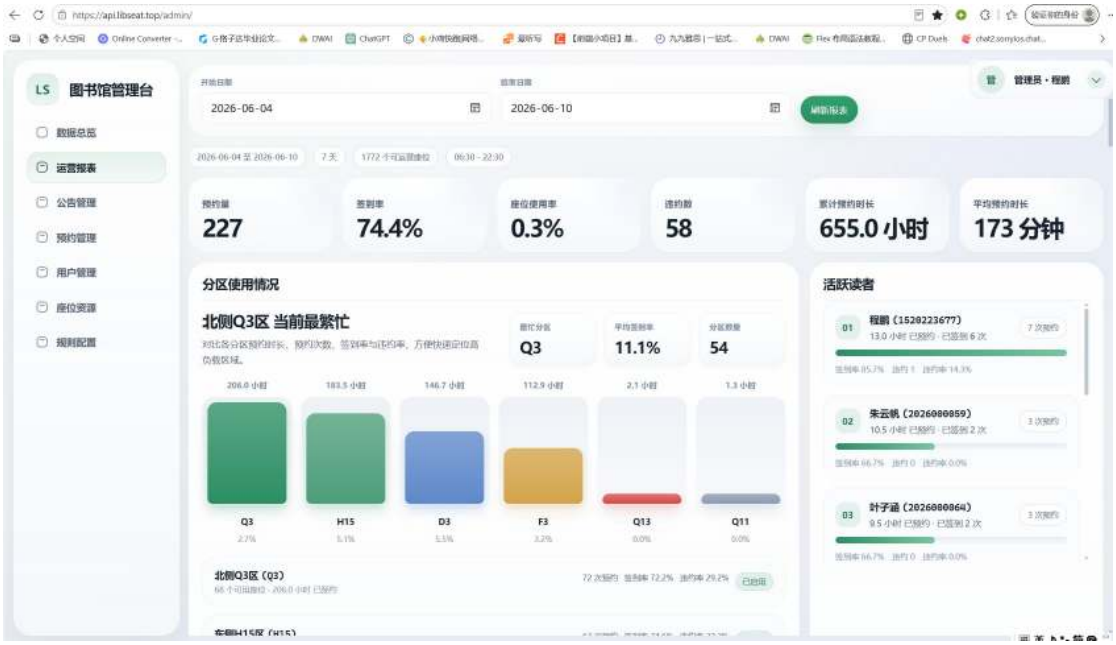


图 5.8 运营报表与分区负载分析界面

图 5.8 表明了管理后台运营报表界面的样子，该页面主要供管理员进行时间范围统计运营分析，管理员利用开始日期和结束日期来筛选统计窗口，可以查看预约量、签到率、座位使用率、违约数、累计预约时长和平均预约时长等指标，页面下方展示分区使用情况和活跃读者列表，分区使用情况用于对比不同区域的预约时长、预约次数、签到率和违约率，活跃读者列表用来识别统计周期内预约频率较高的用户。和图 5.7 后台首页侧重实时运行总览不一样，运营报表更重视在一段时间内的趋势判断和资源负载比较，比如图中能看出西侧 F3 区是当前统计窗口内比较忙的分区，给管理员调整区域开放策略、发布提醒公告和进行异常预约治理提供了依据。

从代码实现看，运营报表页由 admin-web/app.js 中的 loadReports() 请求 /api/admin/reports，并根据 startDate、endDate 和当前管理员身份获取统计窗口内的运营数据；renderReports() 负责把返回结果渲染为指标卡片、分区负载图和活跃读者列表。后端通过 getAdminReports() 聚合 seat_reservation、library_zone、library_seat 和 app_user 等表中的数据，生成 overview、zoneUsage、dailyUsage、hourlyUsage、violationStats 和 topReaders 等结果。前端再由 buildZoneUsageMarkup() 展示分区使用情况，由 buildTopReadersMarkup() 展示活跃读者，使管理员能够从时间范围、空间区域和用户活跃度三个角度观察座位资源使用情况。

5.3.3 公告发布与前台通知联动

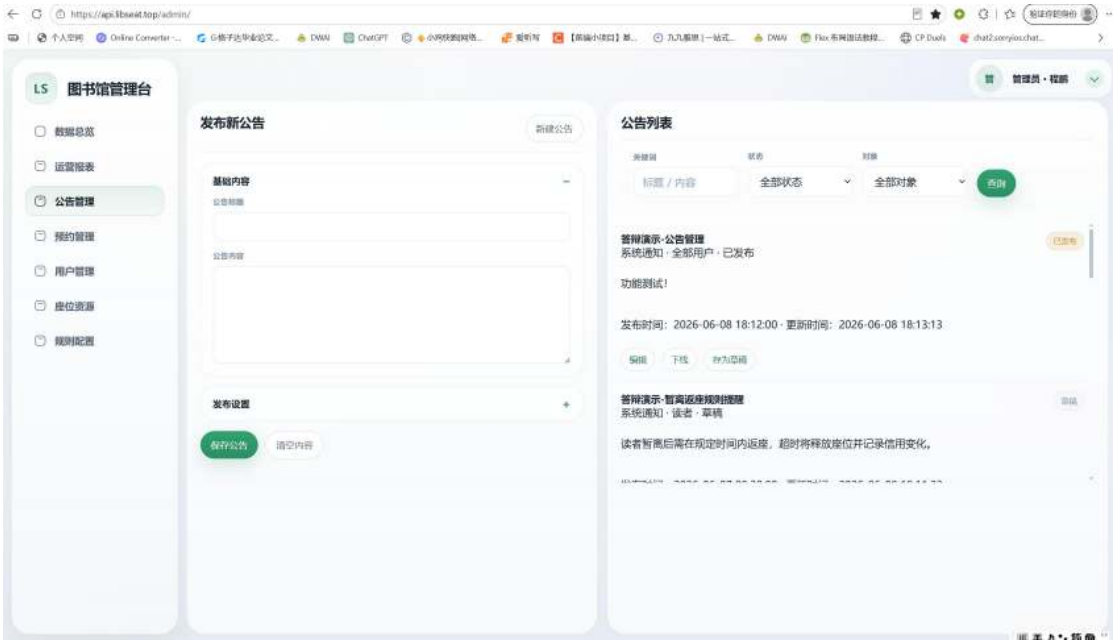


图 5.9 公告发布与维护界面

图 5.9 展示了管理后台的公告管理界面。该页面用于维护面向读者端展示的通知信息，左侧提供公告标题、公告内容、公告类型、发布对象、发布状态和发布时间等编辑入口，右侧提供公告列表、关键词检索、状态筛选和对象筛选功能，管理员可以在同一页面内完成公告新增、编辑、发布、下线和存为草稿等操作。公告内容可以用于临时闭馆提醒、预约高峰提示、签到规则说明和系统试运行通知等场景，能够让后台管理措施及时传达到读者侧。由于小程序首页会展示后台已发布的公告，因此该模块不是单纯的后台文本维护页面，而是系统进行消息触达和规则说明的重要入口。

从实现方式看，公告管理主要由 admin-web/app.js 中的 loadNotices()、collectNoticeForm() 和 updateNoticeStatus() 完成。管理员新增或编辑公告时，前端将标题、内容、发布对象和状态等表单数据提交到 /api/admin/notices，公告上下线则通过 /api/admin/notices/{id}/status 完成；后端在 server/src/index.js 中接收请求，并调用 admin-repository.js 中的公告管理方法写入 system_notice 表。读者端首页通过 getHomeData() 请求 /api/home，后端只返回已发布且仍在有效期内的公告，再由 buildNoticeCards() 渲染到首页通知区域，从而形成后台维护、接口过滤和前台展示的联动关系。

5.3.4 预约管理与异常处理

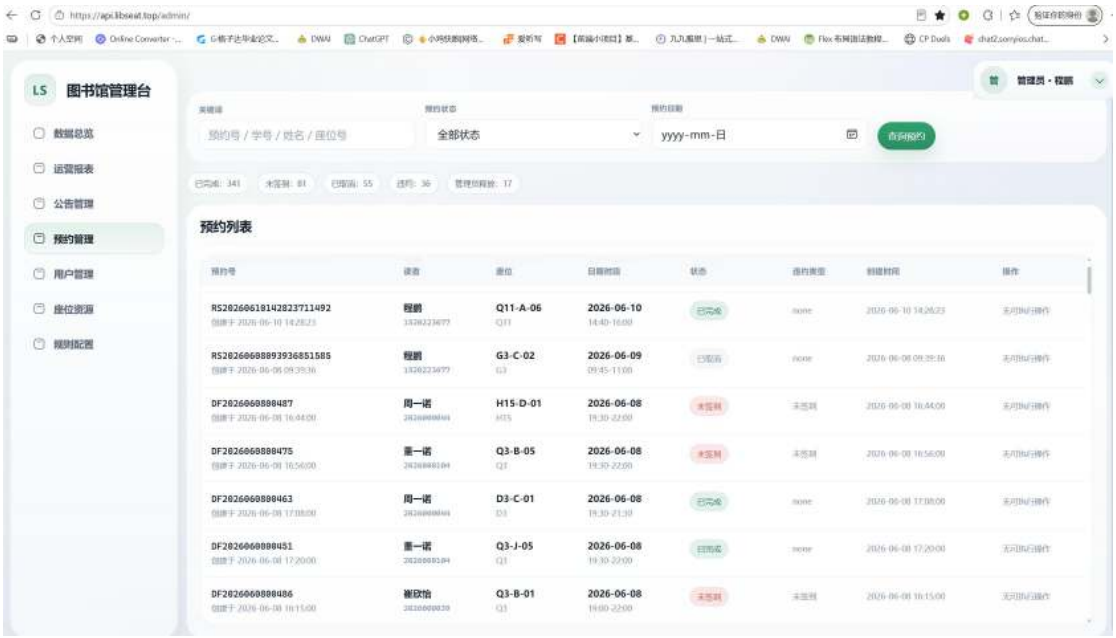


图 5.10 预约记录查询与释放预约界面

图 5.10 展示了管理后台的预约管理界面。该页面用于对读者预约记录进行集中查询和人工干预，管理员可以按照预约号、学号、姓名、座位号、预约状态和预约日期筛选记录，并在预约列表中查看预约号、读者信息、座位编号、预约日期时段、当前状态、违约类型和创建时间等内容。对于仍处于可处理范围内的预约，系统会提供“代为签退”或“释放预约”等操作入口，用于处理异常占用、测试数据清理、读者无法自行签退或管理员巡检发现座位闲置等情况。该模块与小程序端的“当前预约”和“预约记录”页面相互对应，读者端看到的预约状态，最终都来源于后端对预约生命周期的统一维护，因此后台干预后，前台状态也会随接口数据同步更新。

从实现方式看，预约管理页通过 admin-web/app.js 中的 loadReservations() 请求 /api/admin/reservations，并根据关键词、预约状态和日期筛选预约记录；renderReservations() 负责将返回的预约号、读者、座位、时间段和状态渲染到表格中。对于仍可处理的预约，页面通过 buildReservationActionButtons() 显示代签退或释放入口，并分别调用 /api/admin/reservations/{reservationNo}/manual-checkout 和 /api/admin/reservations/{reservationNo}/release。后端在处理这些接口前会同步预约状态，再由 admin-repository.js 完成状态更新和时隙释放，小程序端则通过 /api/reservations/me 读取同一数据源，因此后台干预结果能够同步反馈到读者端。

5.3.5 用户管理与信用治理

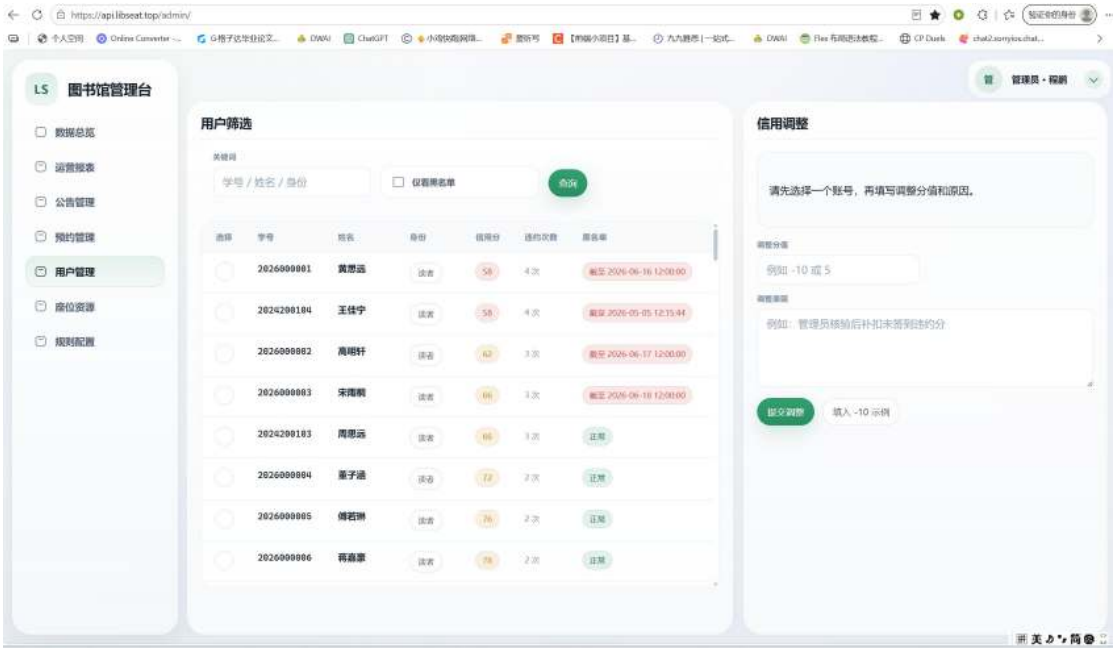


图 5.11 用户筛选与信用调整界面

图 5.11 表示了用户管理和信用治理模块，左侧列表列出了学号、姓名、身份、信用分、违约次数和黑名单状态，并且可以按照关键词或者黑名单状态来进行筛选，用户选中后右侧会显示用于录入调整分值和原因的界面，以便对信用分进行加分或扣分，该模块把读者的履约表现纳入账号维度管理，违约处理所以不再局限于单条预约记录。

从实现方式看，用户管理页通过 loadUsers() 请求 /api/admin/users 获取读者账号、信用分、违约次数和黑名单状态，renderUsers() 将这些信息展示为可筛选列表。管理员选中某一读者后，selectUser() 会把处理对象同步到右侧信用调整区域，提交时通过 /api/admin/users/{studentNo}/credit-adjust 传入调整分值和原因。后端由 adjustAdminUserCredit() 完成信用分更新、黑名单判断和信用日志写入；小程序个人中心则通过 /api/profile/me 读取信用分与信用记录，预约预校验阶段也会在 validateReservationInput() 中检查用户状态和黑名单状态，使信用治理结果能够同时影响展示和预约资格。

5.3.6 座位资源维护

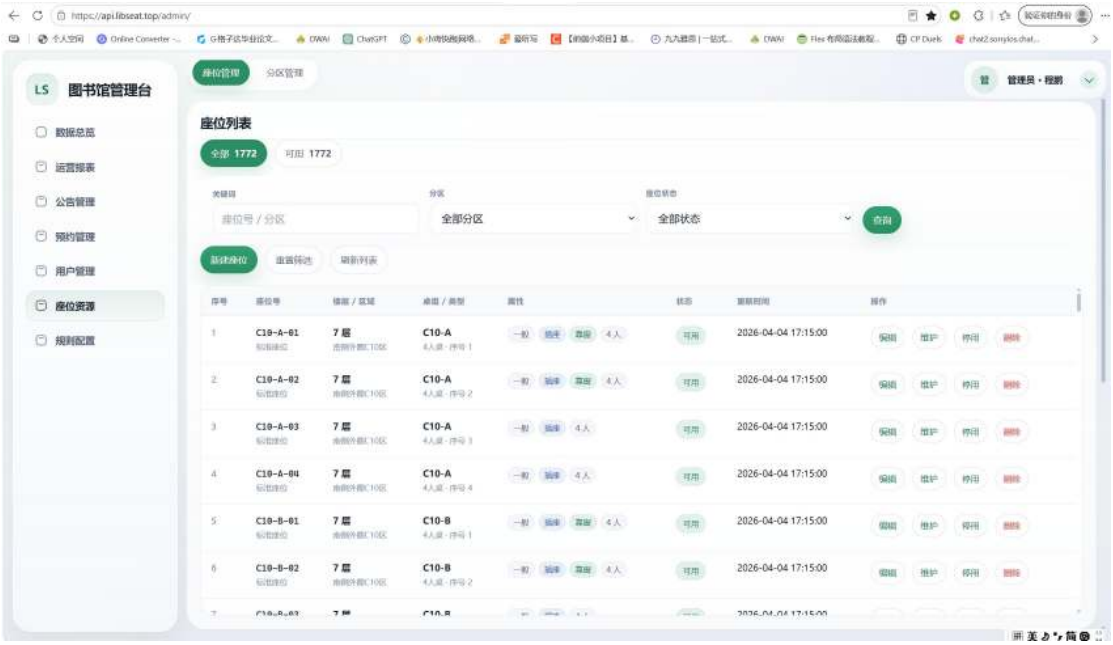


图 5.12 座位资源维护界面

图 5.12 表明座位资源维护页面，后台以列表形式展示座位号、楼层区域、桌组类型、环境属性、座位状态和更新时间，并且提供新增、编辑、维护、停用和删除等操作，页面中的插座、靠窗、四人桌、安静程度等字段会被小程序筛选、分区详情展示和 AI 推荐特征共同使用，所以管理员对座位属性进行维护后，前台候选结果和推荐排序就会发生改变。

从实现方式看，座位资源页通过 loadSeats() 请求 /api/admin/seats，并按照关键词、分区和座位状态返回座位列表；renderResources() 负责展示座位号、楼层区域、桌组类型、环境标签和静态状态。新增或编辑座位时，collectSeatForm() 会收集座位编码、分区、桌组、容量、状态和环境属性等核心信息，并提交到 /api/admin/seats 或 /api/admin/seats/{seatCode}；座位停用、维护和恢复可用则通过状态接口完成。小程序端通过 getSeats() 读取同一套座位主数据，并在选座页中由 buildTableGroups() 聚合为桌组视图，AI 推荐接口也通过 listSeatsFromDb() 获取候选座位，因此后台座位维护会直接影响前台选座范围和推荐候选集合。

5.3.7 规则配置与业务参数统一维护

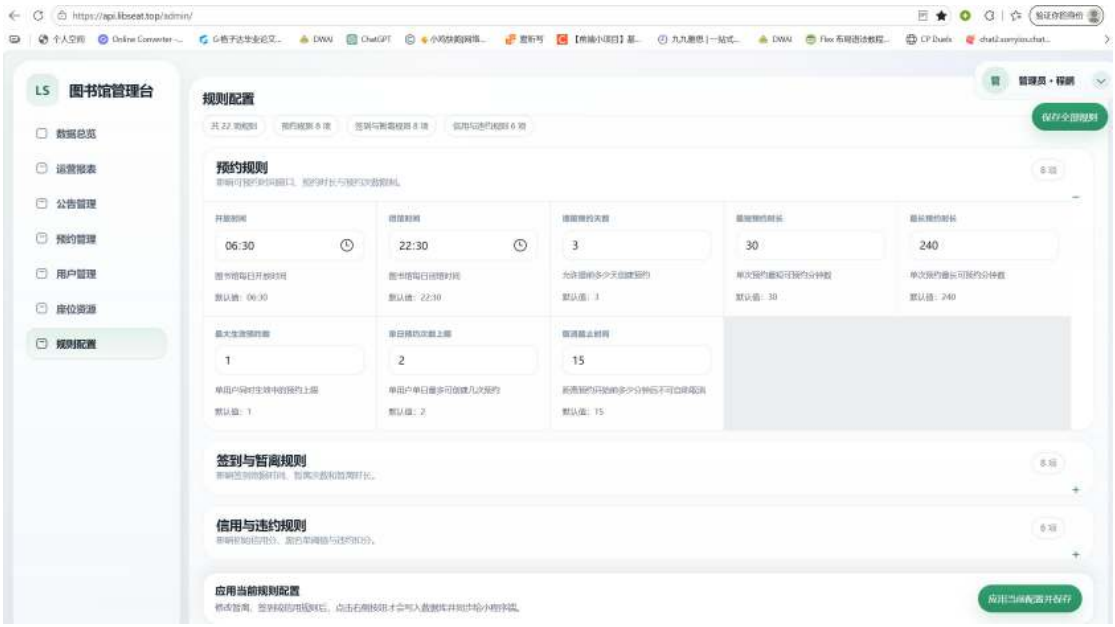


图 5.13 预约与签到规则配置界面

图 5.13 展示了规则配置模块，当前规则依据预约、签到暂离和信用违约这三个方面来进行维护，一共包括 22 项参数，预约组主要包括开放时间、闭馆时间、提前预约天数、最短和最长预约时长、最大生效预约数、单日次数上限、取消截止时间，签到暂离组含有签到时间范围、签到中心经纬度、允许半径、暂离次数和暂离时长，信用组包括初始信用分、黑名单阈值、违约扣分。管理员可以经由页面对这些参数进行调整，而不必直接对程序代码进行修改。

从实现方式看，规则配置页通过 loadConfigs() 请求 /api/admin/configs 获取当前规则项，renderConfigs() 按预约规则、签到与暂离规则、信用与违约规则进行分组渲染。管理员修改参数后，collectConfigFormValues() 收集各配置项并提交到 /api/admin/configs，后端根据 rule-config.js 中的规则定义校验数据类型和取值范围，再写入 system_config 表。小程序端通过 /api/rules 获取规则说明，预约预校验、签到、暂离和返座等接口则通过 getActiveRules() 读取最新配置，并将其传入 validateReservationInput() 等业务函数完成实际约束。

5.4 需求-座位多标签匹配数据集构建

AI 选座数据集将需求座位组合作为样本单位，系统先通过图书馆平面图、分区标注和座位主数据来整理座位属性，接着构造典型需求文本，随后，本文将每一条需求和候选座位进行逐一配对，以生成可以用于模型训练的匹配样本，该设计可以让模型学习某个座位在某条需求下是否合适，而不是给座位固定贴上单一类别。

本文一共形成了 56 条典型用户需求，候选座位数量有 1772 个，在全量组合后构成了 99232 条需求和座位的匹配样本，在对样本进行均衡处理之后，实验利用 34216 条样本进行模型训练、测试工作，每个样本包括 68 项结构化匹配特征，这些特征主要涉及需求侧偏好、座位环境属性、桌组统计特征、分区统计特征、偏好命中项、冲突项和场景评分。系统可以表达用户需要什么和候选座位能否满足需求之间的对应关系。

表 5-1 需求-座位多标签匹配数据集总体情况

数据项	数值或说明
典型用户需求数	56 条
候选座位数	1772 个
全量组合样本数	99232 条
训练用均衡样本数	34216 条
输入特征数	68 项
输出标签数	4 项
样本单位	一条用户需求与一个候选座位组成一条匹配样本
输出标签	深度专注匹配、舒适久学匹配、设备作业匹配、开放协作匹配

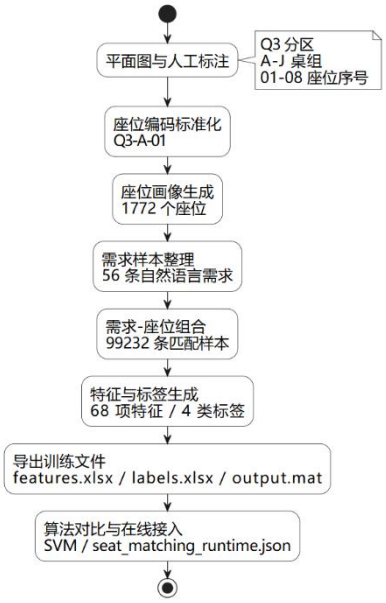


图 5.14 需求-座位多标签匹配数据集构建流程图

图 5.14 展示了需求-座位多标签匹配数据集的构建过程。系统先根据图书馆平面图、分区标注和座位主数据整理座位编码，将分区、桌组和座位序号统一为 Q3-A-01 等标准化形式；随后结合座位位置、环境属性和桌组信息生成座位画像，并整理 56 条自然语言需求样本；需求解析时主要将插座、安静、靠窗、多人桌、远离通道、学习时长等语义线索映射 need_*、scene_* 和 duration_hours_norm 等结构化字段。由于本文的样本单位是一条需求与一个候选座位的组合，因此系

统将 56 条需求与 1772 个座位逐一组合，形成 99232 条匹配样本，再进一步生成 68 项结构化特征和 4 类匹配标签，最终导出 features.xlsx、labels.xlsx 和 output.mat 等训练文件。该流程说明本文的数据集不是简单的座位清单，而是面向推荐模型构建的需求-座位组合样本。

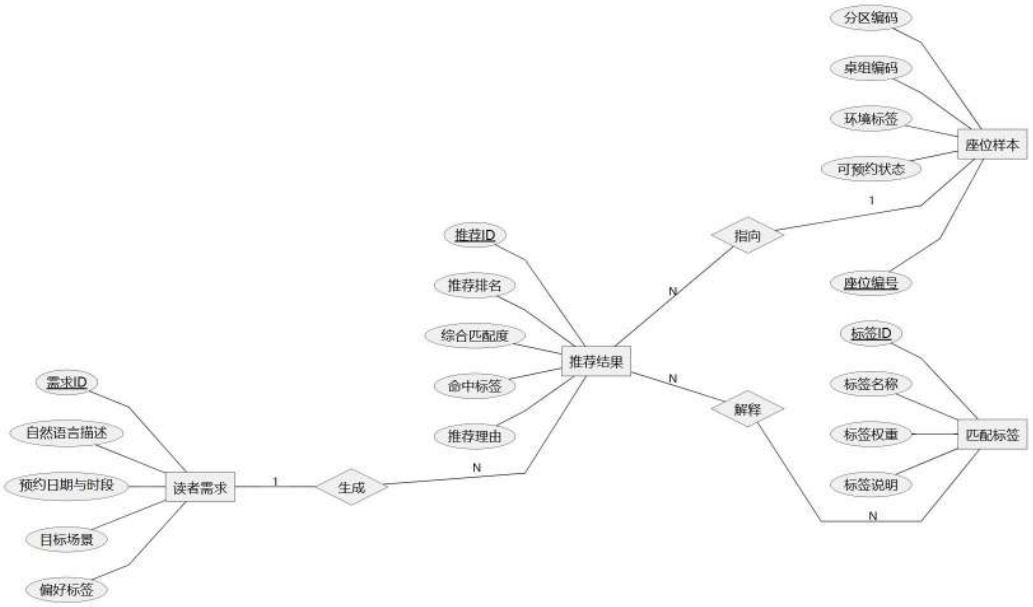


图 5.15 推荐分析实体属性 E-R 图

图 5.15 整理了推荐分析模块包括的四类主要数据对象：第一类是读者需求侧数据集，它保存用户提交的自然语言需求表述、预约日期和时段、预期使用场景及个性化偏好标签，第二是座位基础信息库，该数据库全量记录座位的编号、所属分区代码、所在桌组编码、环境特征标签、实时预约状态，第三是预设匹配标签集合，该集合包括深度专注、舒适长时学习、设备适配办公、开放协同这四类场景属性。第四是推荐结果库，该库存储推荐输出项的排序优先级、综合匹配分值、命中标签及匹配依据，这些数据对象为特征建立、模型训练和推荐结果实时响应提供完整的数据基础。

5.5 匹配标签生成与特征构造方法

本文给每条需求座位样本设置四个标签，分别是深度专注、舒适久学、设备作业和开放协作，同一条样本会同时匹配多个标签，比如座位处于安静状态、靠近窗户并且配有插座，用户同时提出了长时间学习和设备充电的要求时，该样本会同时属于前三类，与单标签分类相比，多标签方式更能够描述读者需求中多个条件同时存在的情况[22]。

在进行特征构造的过程中，本文同时利用了需求侧、座位侧、交叉匹配信息，需求侧代表用户是否需要插座、靠窗、安静、多人桌、书架附近或者远离通道，

座位侧表示候选座位本身是否拥有这些属性，交叉项记录需求是否得到满足，系统也加入冲突项和场景评分，用来描述通道干扰、桌型不匹配等负面因素，把业务信息变成结构化特征，成为模型训练的前提条件[23]。

对于第 i 条用户需求和第 j 个候选座位，本文构造匹配特征向量 x(i,j)，其组成如式（5.1）所示：

$$x_{i,j} = [D_i, S_j, G_j, Z_j, M_{i,j}, C_{i,j}, R_{i,j}]\#(5.1)$$

D(i)表示需求侧特征，S(j)表示座位自身属性，G(j)表示桌组统计特征，Z(j)表示分区统计特征，M(i,j)表示偏好命中项，C(i,j)表示冲突项，R(i,j)表示四类场景评分，该特征向量可以使系统同时描述用户需求、候选座位条件、它们之间的匹配关系。

本文的多标签输出如式（5.2）所示：

$$y_{i,j} = [y_{\text{focus}}, y_{\text{comfort}}, y_{\text{device}}, y_{\text{collab}}]\#(5.2)$$

y_{focus}代表深度专注匹配，$y_{comfort}$代表舒适久学匹配，y_{device}代表设备作业匹配，y_{collab}代表开放协作匹配，这四个标签各自单独存在，一个样本可以同时命中多个标签，本文便使用多标签分类方法来进行建模，该多标签表达方式比只给座位指定单一场景类别要更接近真实的选座场景，主要因为读者一次的需求通常同时包括安静、充电、采光、桌型和空间干扰等很多条件。

5.6 多标签分类模型训练与在线推荐接入

在模型训练阶段，本文以 68 项需求-座位匹配特征作为输入，以深度专注匹配、舒适久学匹配、设备作业匹配和开放协作匹配四个标签作为输出，分别训练逻辑回归、支持向量机 SVM、神经网络 MLP、随机森林、决策树、K 近邻和高斯朴素贝叶斯等模型。为了避免同一条用户需求同时出现在训练集和测试集中，实验采用按需求编号分组的方式划分数据集，从而提高模型泛化评价的可靠性。统计学习中的模型评估通常利用训练误差和测试误差来判断学习方法效果[24]。机器学习中的模型选择需要关注训练误差与泛化误差之间的关系，因此本文通过统一数据集上的多模型比较确定最终部署模型[25]。

在模型训练完成之后，本文把支持向量机模块接入到智能选座系统的应用程序接口里，当用户提交选座需求并选定目标日期和时段之后，后端服务先对用户偏好参数进行解析，同时筛选出当前可以预约的所有座位资源，接着把用户实际需求和候选座位进行匹配，提取出和训练阶段完全相同的 68 维特征向量输入到支持向量机中，在获得四类标签的预测概率结果之后，系统会进一步结合预设的需求权重、偏好匹配奖励值、资源冲突惩罚系数、平台预约规则来计算各个座位的综合匹配分值，最终向用户返回排序前 N 位的推荐座位列表。

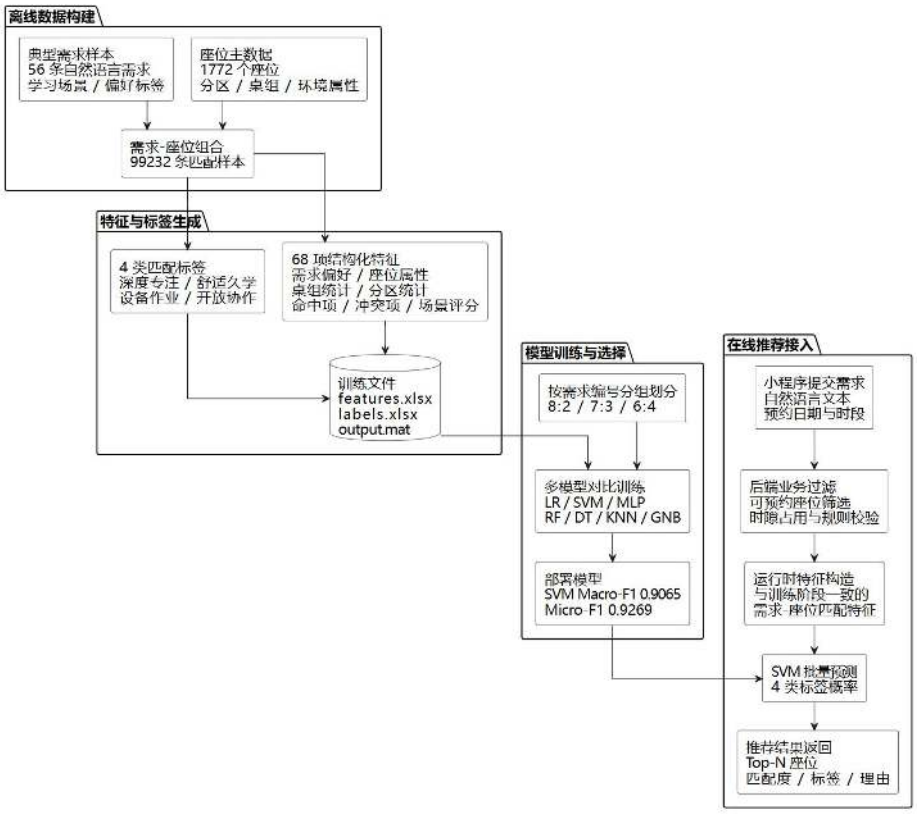


图 5.16 需求-座位匹配模型训练与在线推荐接入结构图

图 5.16 展示了需求-座位匹配模型从离线训练到在线接入的完整链路。离线阶段先整理座位空间数据和典型自然语言需求，将 1772 个座位与 56 条需求组合成 99232 条匹配样本，再生成 68 项结构化特征和 4 类匹配标签，并导出 features.xlsx、labels.xlsx 和 output.mat 用于模型训练；在线阶段由小程序提交用户需求，后端先根据预约日期、时段和座位状态筛出可预约候选座位，再调用 predictSeatRuntime() 对候选座位进行 SVM 批量预测，并把 Top-N 推荐结果返回到 AI 辅助选座页面。

SVM 对需求-座位样本的预测过程如式（5.3）所示：

$$P_{i,j} = f(x_{i,j}) = [p_{\text{focus}}, p_{\text{comfort}}, p_{\text{device}}, p_{\text{collab}}] \#(5.3)$$

预测向量包括的 4 个分量，分别对应该候选座位和深度专注、舒适久学、设备作业、开放协作四类标签的匹配概率，后端没有直接把概率最高的单个标签当作判定结果，而是把当前场景的需求权重、偏好匹配奖励和标签冲突惩罚项加起来算出综合匹配分值，具体的计算规则可以参考公式（5.4）。

$$Score_{i,j} = \sum_{k=1}^{4} \alpha_{k,i} \cdot p_{k,i,j} + Bonus_{i,j} - Penalty_{i,j} \#(5.4)$$

权重项用来反映当前需求在不同学习场景中受到的关注程度，概率项来自于 SVM 多标签预测结果，奖励项会把满足显式偏好座位的得分进行提高，惩罚项

则会将存在冲突特征座位的排序位置进行降低，最终推荐列表的生成方式如式（5.5）所示：

$$RecList_i = TopN\{Score_{i,j} | seat_j \in AvailableSeats(date, time)\} \#(5.5)$$

公式里列出的可以预约的座位集合，是指在特定日期和时段内按照所有预定规则进行验证的备用座位，支持向量机进行的预测和排序工作只会在该集合范围内进行，所以智能选座过程不会超出座位启停状态、时段占用状况、预约时效要求、用户预约权限等已经确定的业务规则限制，该方式可以确保推荐输出的座位和常规人工预约流程得到的座位达到完全相同的合规要求。

5.7 本章小结

本章阐述了系统功能的实现过程、推荐算法接入的过程，小程序端主要依靠首页、空间导览、地图筛选、预约提交、当前预约、AI 选座和个人中心来进行页面组织，后端主要围绕用户认证、首页数据、座位查询、预约预校验、预约状态更新、公告规则读取和推荐接口来组织业务逻辑，管理后台拥有运行总览、运营报表、公告维护、预约处理、用户信用、座位资源和规则配置等功能。上述模块通过统一接口和数据库记录进行连接，前台读者操作、后台管理动作、座位状态变化都保持一致。

在算法实施过程中，本文把图书馆座位推荐任务改变成用户需求和座位资源的多标签匹配问题，系统使用 56 条代表性用户需求和 1772 个候选座位，先生成 99232 组需求和座位配对样本，在完成样本均衡处理后选出 34216 条有效样本以进行模型训练和测试，按照 68 项结构化特征和 4 类匹配标签依次进行标签模式建立、特征工程、模型训练、线上部署。本章的实现过程表明，该系统已经打通了读者端预约功能、后端规则校验模块、后台资源治理方式和智能推荐算法的共同连接路径，同时也为第六章进行系统功能测试、接口性能测试、推荐算法有效性验证提供了实现基础。

第六章　系统测试与实验分析

6.1 测试环境与评价指标

系统测试工作包括四个主要部分，即业务功能验证、规则和安全性测试、性能压力测试、推荐算法适配性实验，功能验证环节主要检查小程序端、后端服务、MySQL 数据库和管理后台这四部分能否共同完成完整的预约流程，规则和安全性测试主要围绕预约冲突校验、用户权限管控和异常输入拦截这三个主要内容进行，性能测试模块主要负责记录系统在高频调用接口和多用户并发预约这两种场景下所表现出的响应情况。算法实验主要是为了验证需求座位多标签匹配模型在 AI 智能选座场景中适用的有效性。

推荐实验使用需求-座位多标签数据集。56 条典型需求与 1772 个候选座位组合后得到 99232 条样本，经过均衡处理后使用 34216 条样本进行训练和测试；每条样本包含 68 项结构化特征和 4 个输出标签。这里的样本不是单个座位，而是某条需求与某个座位的组合，模型需要判断该组合是否命中四类匹配标签。

单一样本通常会关联多个标注标签，本文选取子集准确率、宏平均精确率、宏平均召回率、Macro-F1 值和 Micro-F1 值作为主要评价指标，子集准确率要求样本对应的四个标签预测结果全部正确才算命中，Macro-F1 值主要用来衡量模型对不同类别标签的均衡识别效果，Micro-F1 值更能反映模型的整体预测性能。所以进行模型筛选时不能只用单一准确率指标作为判断依据。

表 6-1 测试与实验环境配置

项目	配置说明
操作系统	Windows 11 本地开发 + Linux 云服务器部署
小程序工具	微信开发者工具 1.03.0
后端运行环境	Node.js，PM2，HTTPS 接口服务
数据库	MySQL
算法实验环境	Python 3.11.5
核心实验库	pandas，scikit-learn，matplotlib
数据规模	56 条典型用户需求、1772 个候选座位、4 个多标签输出
模型评价指标	准确率，Macro-F1，Micro-F1，Top-K 推荐精度，宏平均召回率

6.2 功能测试

功能测试采用黑盒方式，从页面操作和接口输入输出两侧检查系统结果。由于系统包含微信小程序、Node.js 接口、MySQL 数据库和 Web 后台，测试时不只关注页面是否能够打开，还需要核对预约创建、签到签退、后台维护和数据库状态是否能够对应起来，尤其要关注读者端与后台端之间的数据一致性。

读者端测试内容包括登录操作、首页浏览、分区浏览、座位筛选、预约预校验、预约创建、签到签退、历史记录、AI 选座等各项功能，测试过程模拟普通读者从进入小程序到完成一次预约使用的完整路径，本文观察页面数据、状态更新和提示信息是否正确，具体测试结果如表 6-2 所示。

表 6-2 读者端核心功能测试表

测试模块	测试用例	预期结果	实际结果
用户登录	输入正确学号和姓名登录	成功进入系统并返回用户信息	通过
首页概览	进入首页查看公告、分区和快捷入口	首页数据正常显示	通过
分区浏览	选择日期和时间段后查看分区	返回分区列表和可预约数量	通过
座位筛选	按插座、靠窗、安静等条件筛选座位	返回符合条件的座位列表	通过
预约预校验	提交合法日期、时间段和座位	返回校验通过结果	通过
预约创建	预校验通过后提交预约	生成预约号并显示预约成功	通过
当前预约	进入当前预约页面查看记录	正确显示预约座位、日期和状态	通过
取消预约	取消未开始的预约	预约状态更新并释放占用	通过
签到签退	在预约时间内执行签到和签退	状态正常流转并显示反馈	通过
个人中心	查看信用分、预约统计和历史记录	用户信息和信用状态正常显示	通过
AI 辅助选座	输入自然语言学习需求	返回推荐座位和推荐依据	通过

由表 6-2 可以看出，读者端主要功能均能得到符合预期的结果。系统能够支持用户完成从首页查看、座位筛选、预约提交到状态反馈的完整流程，预约创建后能够在当前预约页面正确回显，取消预约后状态也能同步更新。AI 辅助选座功能能够根据用户输入的自然语言需求，调用后端推荐接口完成候选座位筛选、模型预测和推荐理由生成，返回结果可以直接进入小程序端的选座流程，说明推荐模块已经不只是离线实验结果，而是接入了实际业务链路。

服务端接口测试重点检查请求参数、返回数据和业务状态。预约预校验、预约创建、座位时间轴和 AI 推荐接口是测试中的关键对象，因为这些接口直接决定前端能否完成预约流程；如果接口只返回页面数据但不能正确更新业务状态，系统仍然不能算通过测试。具体测试结果见表 6-3。

表 6-3 服务端 RESTful API 功能测试表

测试模块	测试接口或功能	测试内容	测试结果
健康检查	GET /api/health	检查服务运行状态和数据库连接情况	通过
用户登录	POST /api/auth/login	使用学号和姓名登录，返回当前用户信息	通过
首页数据	GET /api/home	返回座位总数、热门分区、公告和快捷入口数据	通过
规则查	GET /api/rules	返回开放时间、预约时长、签到窗口和	通过

询		信用规则	
分区查询	GET /api/zones	按日期、时段和筛选条件返回分区匹配数量	通过
座位查询	GET /api/seats	返回座位环境标签、可预约状态和推荐排序	通过
座位时间轴	GET /api/seats/{seatId}/schedule	返回指定座位当日占用时间段	通过
预约预校验	POST /api/reservations/preview	校验用户、座位、日期、时段、冲突和规则限制	通过
预约创建	POST /api/reservations	通过事务创建预约并返回预约编号	通过
预约取消	POST /api/reservations/{no}/cancel	取消未开始预约并释放座位占用	通过
签到签退	POST /api/reservations/{no}/checkin、checkout	验证签到窗口和预约状态流转	通过
AI 选座	POST /api/assistant/seat	解析自然语言需求并返回 Top 推荐座位	通过

表 6-3 显示，健康检查、首页、规则、分区和座位查询接口能够正常返回基础数据，预约相关接口能够完成预校验、创建、取消、签到和签退，AI 选座接口也能够返回推荐结果。由此可以说明，后端接口已经覆盖小程序端主要业务和推荐功能的数据需求。

后台管理测试用来确认管理员操作确实影响前台业务，测试过程中需要检查后台页面是否可以打开，并且要核对公告发布是否显示在小程序首页上，座位状态修改是否会影响到可预约结果，规则调整是否进入预校验，信用调整是否回显到个人中心，测试的具体结果如表 6-4 所示。

表 6-4 后台管理功能测试表

后台模块	测试内容	预期结果	实际结果
运行总览	查看总座位数、当前预约、今日预约和使用率	统计指标正常显示	通过
运营报表	按日期查看预约量、签到率、使用率和违约数	图表与列表可正常刷新	通过
公告管理	新增、编辑、发布、下线公告	小程序首页同步展示已发布公告	通过
预约管理	按预约号、学号、状态和日期筛选预约记录	查询结果准确，可处理异常预约	通过
用户信用	调整用户信用分并记录原因	信用分变化写入日志并同步到个人中心	通过
座位管理	新增、编辑、停用和恢复座位	座位状态影响前台可预约结果	通过
规则配置	修改预约时长、提前预约天数、暂离上限	规则变化同步影响预约预校验结果	通过

表 6-4 表明，后台管理端可以对公告、预约、用户、座位和规则等重要工作进行管理，公告发布后会同步显示在小程序首页上，信用调整后会在个人中心中体现出来，座位状态发生改变会影响前台的可预约结果，规则配置也进入预约预校验流程，后台数据展示页面并不是单独存在的，它和读者端预约流程形成了联动关系。

6.3 业务规则与访问控制测试

业务规则测试主要用于验证系统在异常输入和边界条件下是否能够正确拒绝不合法操作。图书馆预约系统具有较强的规则约束性，如果系统只在正常路径下能够创建预约，而不能处理时间冲突、座位停用、黑名单限制等情况，就会影响预约公平性和系统可靠性。因此，本文围绕预约时间、开放时间、账号冲突、座位冲突、签到窗口、暂离时长和信用限制等场景进行测试，结果如表 6-5 所示。

表 6-5 数据库与业务规则测试表

测试规则	测试方法	预期结果	实际结果
预约时间合法性	提交结束时间早于开始时间的预约	预校验失败并提示时间不合法	通过
开放时间限制	提交闭馆后的预约时段	系统拒绝创建预约	通过
最长预约时长	提交超过 240 分钟的预约	提示超过单次预约时长限制	通过
提前预约限制	提交超过 3 天后的预约日期	提示不在允许预约窗口内	通过
同账号冲突	同一用户预约同一时段其他座位	提示该账号已有时间冲突	通过
同座位冲突	多个用户预约同一座位同一时段	只允许首个有效请求成功	通过
座位状态限制	预约停用或维护中的座位	系统拒绝预约	通过
签到窗口限制	在签到窗口外执行签到	提示未到签到时间或已超时	通过
暂离时长限制	暂离超过允许时间未返座	标记暂离超时并进入违规处理	通过
信用限制	黑名单用户发起预约	预校验失败并提示账号受限	通过

由表 6-5 可以看出，系统能够对主要业务规则进行有效约束。对于时间不合法、超出开放范围、超过最长预约时长、同账号冲突和同座位冲突等情况，系统均能够在预校验或创建阶段给出拒绝结果；对于停用座位、签到窗口外签到、暂离超时和黑名单用户预约等场景，系统也能够按照规则进行限制。这说明预约规则不仅停留在前端提示层面，也已经进入后端校验和数据库状态控制过程。

访问控制与安全性测试主要用于验证不同角色是否只能访问其权限范围内的功能，并检查非法参数、越权操作和重复提交是否会造成异常结果。本文系统中普通读者和管理员具有不同权限，普通读者主要使用小程序端，管理员则可以进入后台进行资源维护和规则配置。具体测试结果如表 6-6 所示。

表 6-6 访问控制与安全性测试表

测试项	测试方法	预期结果	实际结果
未登录访问	不携带身份信息访问个人预约接口	返回未授权提示	通过
普通用户访问后台	使用读者账号访问 /api/admin/* 接口	返回无权限提示	通过
越权取消预约	使用其他用户身份取消非本人预约	系统拒绝操作	通过
非法座位编号	提交不存在的 seatId	返回座位不存在或参数错误	通过
非法时间参数	提交错误日期或时间格式	返回参数校验失败	通过
SQL 注入输入	在学号、姓名、筛选参数中输入注入字符串	参数化查询不执行恶意语句	通过
重复提交	连续点击预约提交按钮	系统只保留一条有效预约或返回冲突	通过

由表 6-6 可以看出，系统能够对未登录访问、普通用户访问后台、越权取消预约和非法参数提交等行为进行限制。对于重复提交和 SQL 注入输入，系统也能够通过参数校验和冲突控制避免产生异常数据。虽然本文系统仍属于毕业设计原型系统，但基本访问控制和输入校验已经能够满足当前业务测试要求。

6.4 性能与并发测试

性能测试主要用来验证系统在常见访问场景下所具有的响应能力，并发测试则用来观察多个用户同时进行查询或者预约时系统是否可以保持稳定，图书馆预约系统在考试周、自习高峰期会出现集中访问，本文选择首页查询、分区查询、座位列表查询、同座位抢约和同账号重复预约等场景进行测试，结果如表 6-7 所示。

表 6-7 并发预约压力测试表

测试场景	并发规模	测试内容	结果分析
首页查询	50 个并发请求	同时刷新首页资源摘要和公告	请求均能返回，数据展示正常
分区查询	50 个并发请求	同时查询指定日期和时间段下的分区匹配数量	返回结果稳定，无明显异常
座位列表查询	80 个并发请求	同时查询热门分区座位状态	座位状态可正常返回
同座位抢约	30 个并发请求	多用户同时预约同一座位同一时段	仅一个请求成功，其余返回冲突
同账号重复预约	20 个并发请求	同一账号并发预约多个重叠时段	系统拦截重复有效预约

由表 6-7 可以看出，系统在查询类并发场景下能够保持正常返回，在预约类并发场景下也能够通过冲突校验避免重复有效预约。特别是同座位抢约和同账号

重复预约两个场景，测试重点不在于让所有请求成功，而是验证系统能否正确返回成功、冲突或规则失败结果。测试结果说明系统具备基本并发控制能力。

为了进一步观察核心接口的响应情况，本文对健康检查、首页、规则、分区、座位查询、预约预校验、预约创建和 AI 选座等接口进行了多次请求统计。接口响应时间测试结果如表 6-8 所示。

表 6-8 核心接口响应时间测试表

接口名称	测试次数	平均响应时间	最大响应时间	测试结论
GET /api/health	50	1 ms	2 ms	响应正常
GET /api/home	50	11 ms	14 ms	响应正常
GET /api/rules	50	1 ms	2 ms	响应正常
GET /api/zones	50	29 ms	74 ms	响应正常
GET /api/seats	50	52 ms	66 ms	响应正常
POST/api/reservations/preview	50	4 ms	8 ms	响应正常
POST /api/reservations	50	8 ms	34 ms	响应正常
POST /api/assistant/seat	30	2420 ms	2490 ms	原型可接受，后续需优化

由表 6-8 可以看出，健康检查、首页、规则查询、分区查询和座位查询等普通业务接口平均响应时间均低于 100 ms，能够满足读者端页面加载和基础数据展示需要；预约预校验和预约创建接口虽然涉及规则校验、冲突检查和数据库写入，但在本次测试中平均响应时间分别为 4 ms 和 8 ms，说明当前原型系统的预约写入流程响应较快。AI 选座接口平均响应时间为 2420 ms，最大响应时间为 2490 ms，明显高于普通接口，主要原因是该接口需要完成自然语言偏好解析、候选座位筛选、运行时特征构造、SVM 实时预测和推荐排序。该结果表明，当前 AI 辅助选座功能已经接入真实模型推理流程，能够满足毕业设计原型系统的交互演示需要；若后续进入正式部署，可通过模型常驻服务、候选集裁剪和推荐结果缓存等方式继续降低在线调用耗时。

6.5 推荐算法实验结果与分析

在完成功能测试、业务规则测试和接口性能测试后，本文进一步对 AI 辅助选座模块中的推荐算法进行实验分析。与传统的座位分类不同，本文将推荐任务建模为需求-座位多标签匹配问题，即将一条用户需求与一个候选座位组合为一条样本，由模型判断该样本是否命中深度专注匹配、舒适久学匹配、设备作业匹配和开放协作匹配四类标签。实验数据由 56 条典型用户需求和 1772 个候选座位组合生成，共形成 99232 条需求-座位匹配样本。经过样本均衡处理后，本文使用 34216 条样本进行模型训练与测试，每条样本包含 68 项结构化匹配特征和 4 个多标签输出。

本节按照实验步骤对推荐模型进行分析：首先观察四类标签的分布情况，然后比较七种模型，接着测试不同训练测试比例，最后分析 SVM 的逐标签效果、Top-N 推荐命中率和预测耗时，评价指标有子集准确率、宏平均精确率、宏平均召回率、Macro-F1、Micro-F1 和 Top-K 推荐精度，Macro-F1 主要用来观察各标签之间的均衡识别能力。

6.5.1 数据集与标签分布分析

在多标签建模模式中，一个样本可以对应多个匹配结果，例如安静、靠窗且带有插座的座位，在进行长时间学习和设备充电时，会同时满足深度专注、舒适久学和设备作业的要求，在训练之前，本文先查看四类标签的正样本分布，以此来判断是否存在明显的不均衡情况，因为各个标签的数量存在差异，后续实验在使用 F1、精确率和召回率进行分析的同时，也对准确率进行了观察。

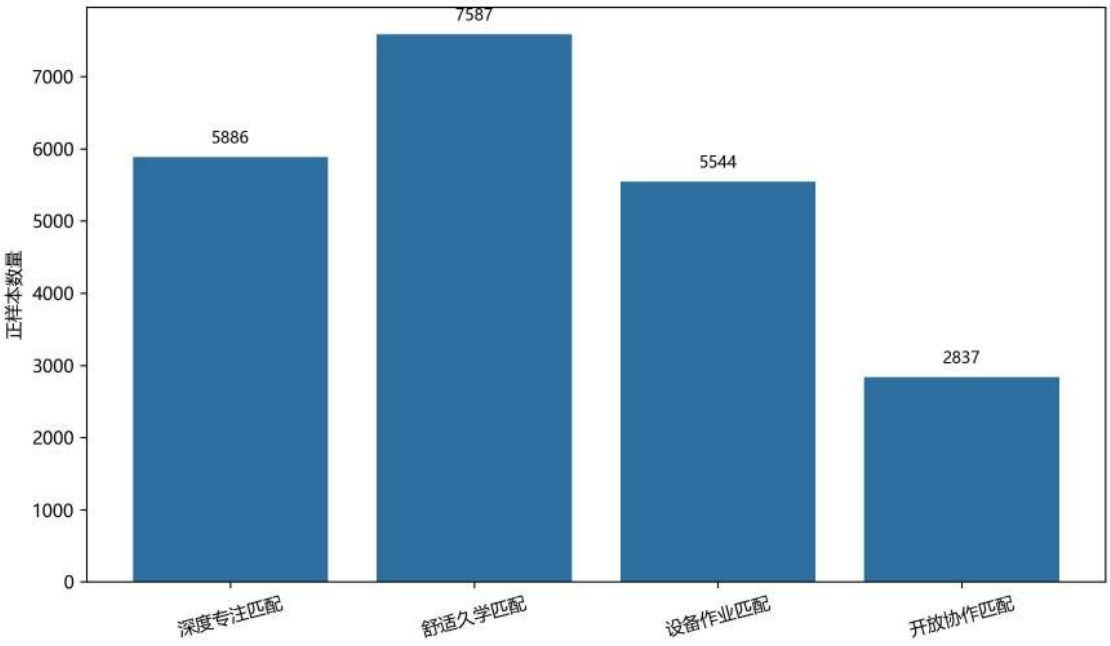


图 6.1 训练样本多标签正样本分布图

图 6.1 表明，四类匹配标签都有一定数量的正样本，不过不同标签之间并不完全均衡，这表明本文的推荐任务不是普通单标签分类，也不是简单地给每个座位分配唯一类别，而是要求模型同时判断一个需求座位样本在多个匹配方面上的适配情况，就该任务而言，仅依靠普通准确率来评价会忽视少数标签或者标签间不均衡所产生的影响，所以，本文在后面的实验里同时使用 Macro-F1、Micro-F1、召回率和这些指标，对模型的整体效果和逐标签识别能力进行综合评价。

6.5.2 多模型对比实验

为选取适合该系统在线推荐场景的模型，本文在相同的需求和座位多标签匹配数据集上，对逻辑回归、支持向量机 SVM、神经网络 MLP、随机森林、决策树、K 近邻和高斯朴素贝叶斯这七种算法进行对比实验，各模型使用相同的特征矩阵和标签矩阵，并按照需求编号分组的方法来划分训练集和测试集，使同一条需求不会同时出现在训练集和测试集中，从而提高实验评价的可靠性。

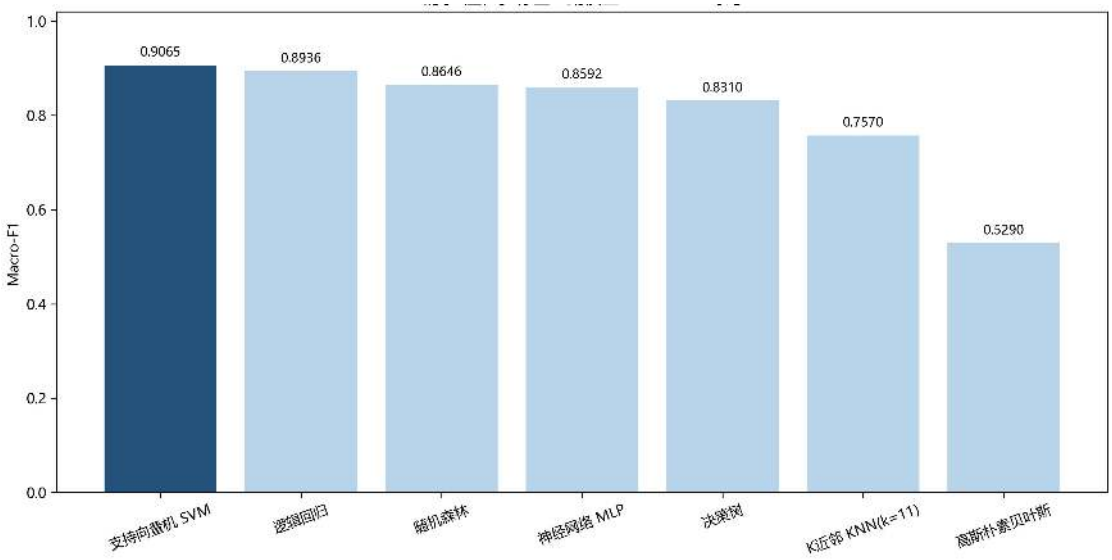


图 6.2 多模型 Macro-F1 对比图

图 6.2 显示，SVM 的 Macro-F1 为 0.9065，在七种模型中最高；逻辑回归为 0.8936，说明本文构造的结构化特征对线性模型也较友好；随机森林和 MLP 分别为 0.8646 和 0.8592，表现相对稳定但低于 SVM；决策树、K 近邻和高斯朴素贝叶斯较弱，尤其朴素贝叶斯受特征独立性假设影响较大。综合来看，SVM 更适合作为在线推荐主模型。

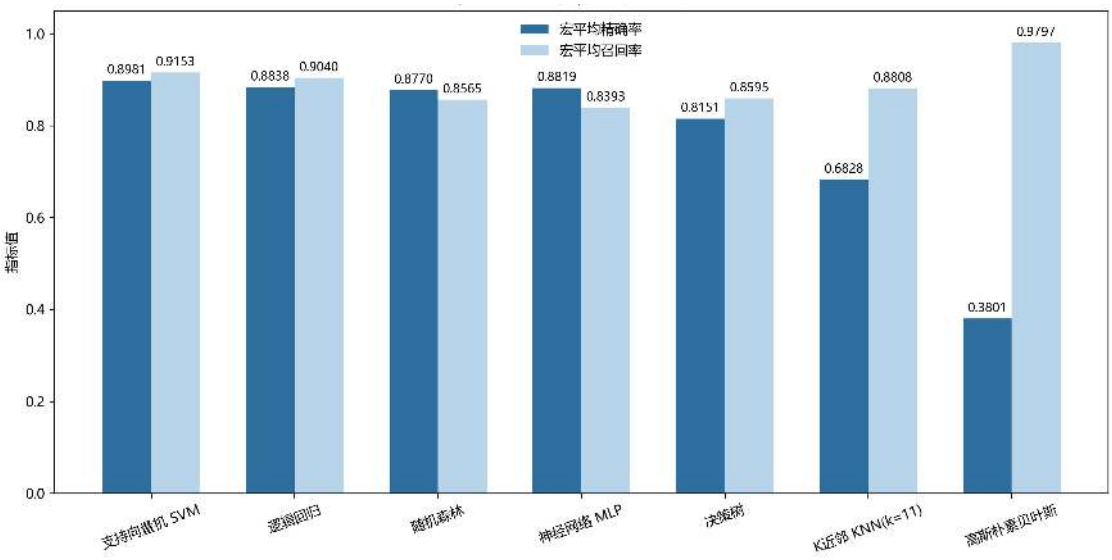


图 6.3 多模型精确率与召回率对比图

图 6.3 对比了宏平均精确率和宏平均召回率，SVM 的宏平均精确率是 0.8981，宏平均召回率为 0.9153，这两个指标比较均衡，所以 Macro-F1 最优，高斯朴素贝叶斯模型的召回率比较高，但是其精确率只有 0.3801，容易将数量较多的样本误判为正类，K 近邻方法也会出现精确率不够的情况，本文选用 SVM，主要因为该模型在精确率、召回率和 F1 值上的整体表现较好。

6.5.3 训练测试比例实验

为检查模型稳定性，本文设置 8:2、7:3 和 6:4 三种训练测试比例。划分时按需求编号分组，即同一条用户需求生成的样本不会同时出现在训练集和测试集中。这样可以减少同源需求带来的相似样本泄漏，使测试结果更接近模型面对新需求时的表现。

表 6-9 SVM 在不同训练测试比例下的实验结果

训练集:测试集	样本划分说明	训练需求数	测试需求数	SVM Macro-F1	SVM Micro-F1
8:2	8:2 划分	45	11	0.9065	0.9269
7:3	7:3 划分	39	17	0.8958	0.9140
6:4	6:4 划分	34	22	0.9038	0.9156

由表 6-9 可以看出，当训练测试比例发生变化时，SVM 的 Macro-F1 始终保持在 0.89 以上。其中，训练集:测试集为 8:2 时 Macro-F1 达到 0.9065；当测试集扩大到 7:3 时，Macro-F1 略降至 0.8958；当进一步调整为 6:4 时，Macro-F1 回升至 0.9038。整体来看，SVM 在不同划分比例下没有出现明显失稳，说明该模型对本文构造的需求-座位匹配特征具有较好的泛化能力。

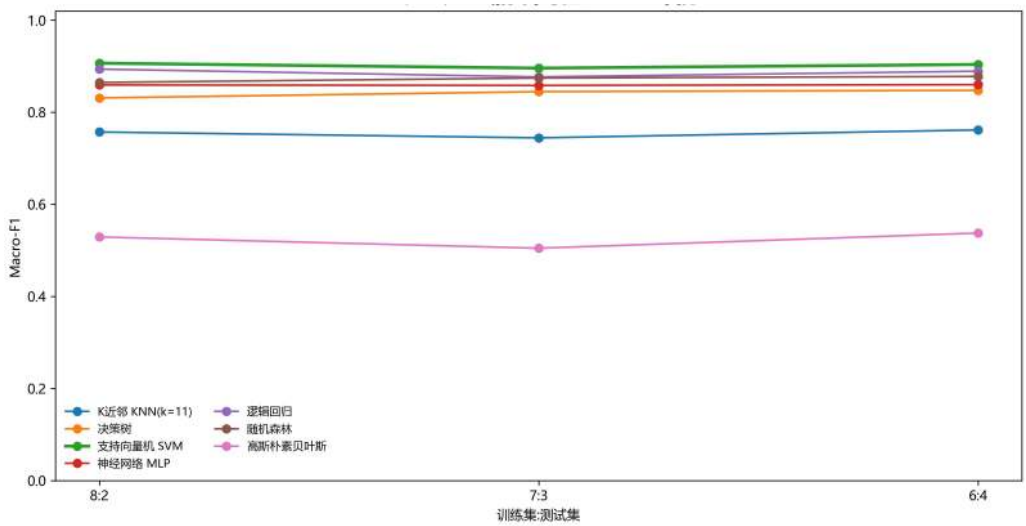


图 6.4 不同训练测试比例下 Macro-F1 趋势图

图 6.4 表明在三种训练测试比例下 Macro-F1 的数值，SVM 在 8:2、7:3 和 6:4 下都维持着比较高的数值，整体数值比大部分模型都要高，逻辑回归保持稳定，但数值略低于 SVM，随机森林和 MLP 波动不大，不过指标仍有差距，K 近

邻和朴素贝叶斯数值始终较低，分析表明，当训练样本比例发生改变时，SVM 仍然可以维持较好的识别效果。

6.5.4 SVM 逐标签效果分析

在把 SVM 当作最终推荐模型之后，本文接着分析了它在四类匹配标签上的具体识别效果，多标签分类任务里各个标签的样本数量和识别难度并不相同，只看总体 Macro-F1 无法清楚地说明模型在不同学习需求上的表现，本文统计了 SVM 在深度专注匹配、舒适久学匹配、设备作业匹配和开放协作匹配这四个标签上的 Precision、Recall 和 F1-score。

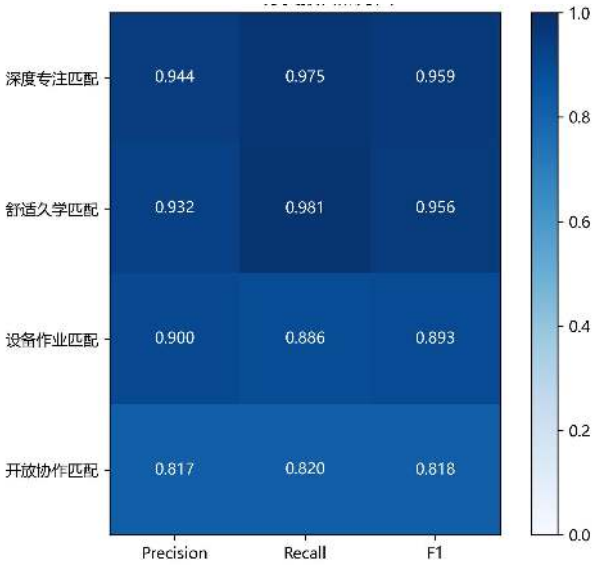


图 6.5 SVM 各匹配标签分类报告热力图

图 6.5 表明，SVM 在深度专注和舒适久学上的 F1 分别为 0.9590 和 0.9558，表明模型对安静、低干扰、靠窗和长时间学习等特征识别较好，设备作业的 F1 为 0.8930，其对插座和设备学习相关需求也相对稳定，开放协作的 F1 为 0.8182，低于其他标签，主要因为协作样本数量较少，并且该标签和桌组容量、多人桌、空间开放性等组合特征之间的关系比较复杂。

SVM 可以识别四类匹配标签，对专注学习、舒适久学和设备作业的需求有较好的表现，支持小程序 AI 辅助选座的主要推荐场景，就开放协作类标签而言，后续可以通过增加协作学习需求样本、细化桌组环境特征、优化标签生成规则来进一步提高识别效果。

6.5.5 Top-N 推荐结果验证

之前的实验评价了多标签预测的效果，小程序端最终展示排序后的座位列表，因此本文从 Top-N 推荐的角度验证 SVM，测试过程中，系统先筛选出可以预约的座位，接着运用 SVM 算法计算四类概率，再将需求权重、偏好命中率和冲突

惩罚相结合，以此获得综合匹配度，最终统计推荐列表中前 K 个座位中强推荐命中的比例。

表 6-10 SVM Top-K 推荐精度统计

Top-K	强推荐 Precision@K	说明
Top-5	0.9636	前 5 个推荐座位中强推荐命中率最高
Top-10	0.9273	推荐列表扩大后仍保持较高命中率
Top-20	0.8864	候选范围继续扩大后推荐精度自然下降但仍可用

由表 6-10 可以看出，SVM 在 Top-N 推荐验证中具有较好的排序效果。当 K=5 时，强推荐 Precision@5 达到 0.9636，说明推荐列表最靠前的座位大多能够满足当前用户需求；当 K 扩大到 10 时，Precision@10 仍达到 0.9273；当 K 进一步扩大到 20 时，Precision@20 为 0.8864。随着推荐列表长度增加，系统纳入的候选座位范围更宽，后部座位的匹配强度自然下降，因此 Precision@K 略有降低是合理现象。

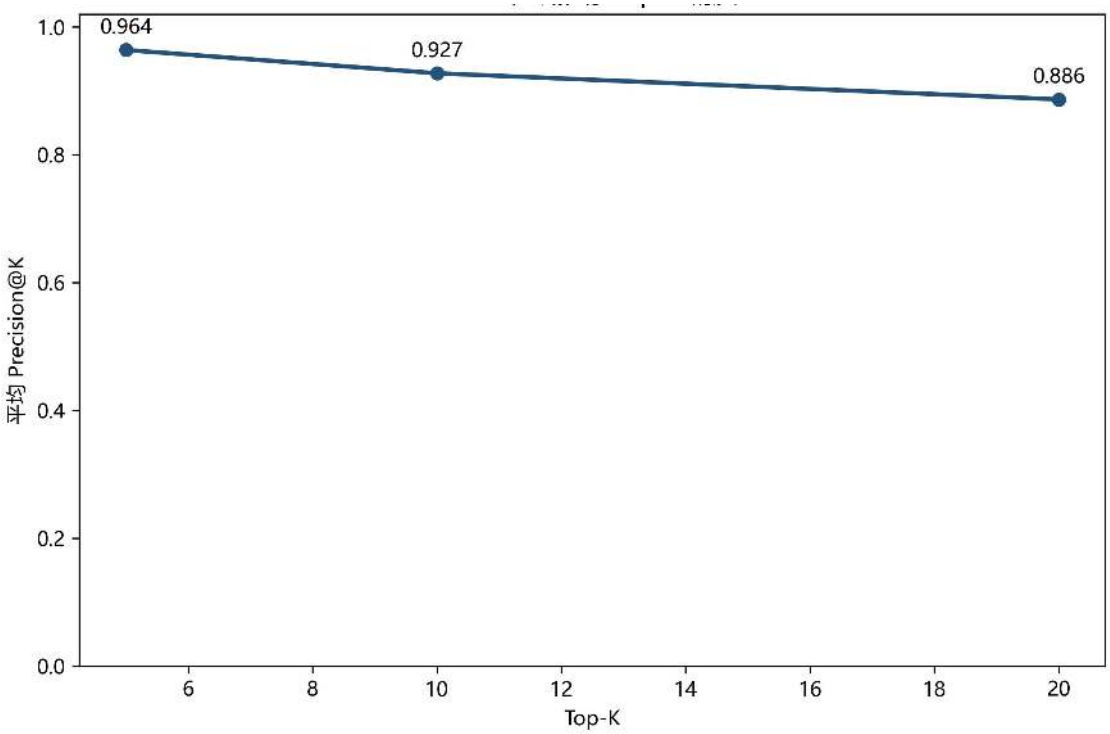


图 6.6 SVM Top-K 推荐精度图

图 6.6 展示了不同 K 值下的推荐命中情况。SVM 在 Top-5 和 Top-10 范围内都保持较高强推荐命中率，说明更符合需求的座位能够排在列表前部。实际使用时，读者通常只会重点查看前几项候选座位，因此 Precision@5 和 Precision@10 比完整列表准确率更能反映 AI 选座的实用价值。

为了进一步展示推荐结果的具体排序效果，本文选取一条测试需求，对其 Top-10 推荐座位的匹配度进行可视化分析。

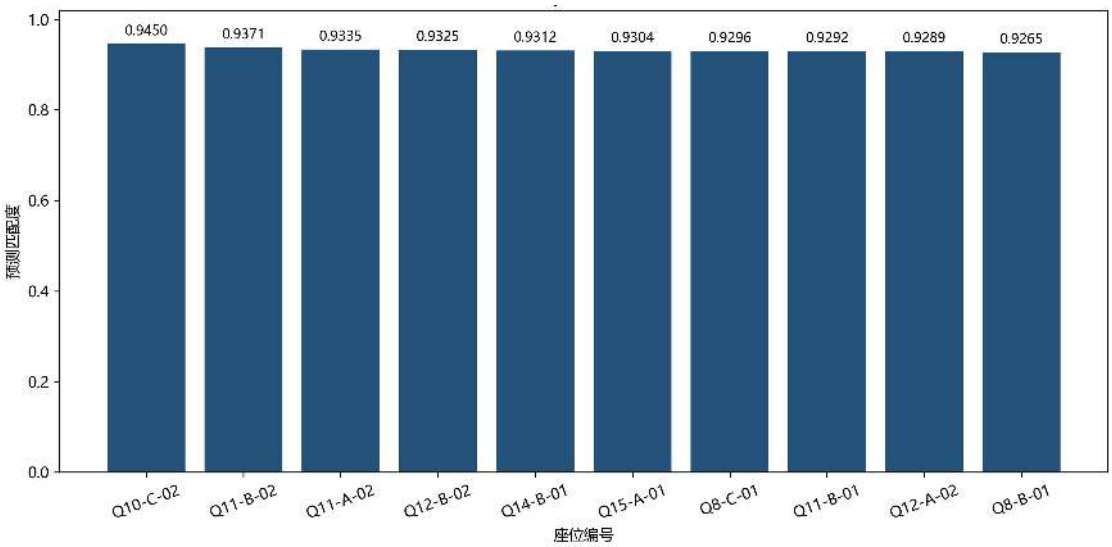


图 6.7 SVM 推荐样例 Top-10 匹配度图

由图 6.7 可以看出，在该测试需求下，推荐列表前 10 个座位的综合匹配度整体较高，排名靠前的座位之间得分差异较小，说明这些座位都能较好满足用户需求。该案例将前文的分类指标进一步转化为具体推荐结果，说明系统最终呈现给读者的不是抽象标签，而是经过模型预测和业务规则排序后的候选座位列表，并能够通过匹配度和推荐理由辅助用户完成选座决策。

6.5.6 模型部署适用性分析

在线推荐场景不仅要求模型具有较高的离线评价指标，还要求模型在实际接口调用中具有可接受的预测开销。AI 辅助选座接口在收到用户需求后，需要先筛选可预约座位，再对多个候选座位批量构造特征并调用模型预测。如果模型预测耗时过高，即使离线指标较好，也会影响小程序端的交互体验。因此，本文进一步比较不同模型的训练耗时和预测耗时，并结合 Macro-F1 指标分析模型部署适用性。

表 6-11 多模型效果与运行耗时对比

模型	Macro-F1	训练耗时/s	预测耗时/s	部署适用性说明
支持向量机 SVM	0.9065	3.1779	0.2570	指标最高且预测耗时较低，适合在线推荐
逻辑回归	0.8936	2.1608	0.2705	速度较快，但指标略低于 SVM
随机森林	0.8646	6.3083	1.7136	指标和预测耗时均不占优
神经网络 MLP	0.8592	23.4148	0.7470	训练耗时较长，效果低于 SVM
决策树	0.8310	1.2501	0.6807	速度可接受，但整体效果不足
K 近邻 KNN	0.7570	0.3298	9.7632	预测耗时过高，不适合实时推荐
高斯朴素贝叶斯	0.5290	0.1483	1.2557	召回率高但误判多，综合效果较弱

表 6-11 表明，SVM 在得到最高 Macro-F1 时，模型预测时间为 0.2570 s，主要因为该模型在离线模型测试中预测开销较低，逻辑回归的训练和预测速度也比

较快，但是 Macro-F1 略低于 SVM，随机森林和 MLP 的整体效果没有超过 SVM，而且训练或者预测开销更高，K 近邻模型的训练时间较短，但预测阶段需计算测试样本与训练样本之间的距离，预测耗时达到 9.7632 s，不适合在线推荐场景。应当说明，表 6-11 统计了模型训练和预测环节，实际接口调用会叠加候选座位筛选、运行时特征构造和推荐排序等步骤，所以应当结合表 6-8 共同判断在线部署效果。

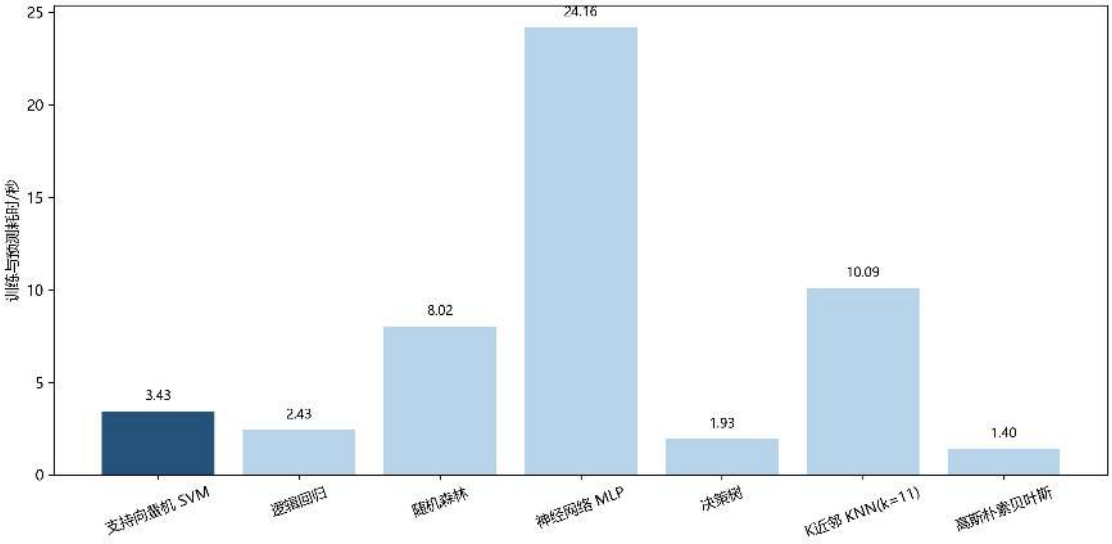


图 6.8 多模型训练与预测耗时对比图

图 6.8 对比了不同模型在训练和预测上所花费的时间，训练过程在离线阶段进行，对用户等待时间产生的影响比较小，预测环节会参与到在线推荐调用中，所以有必要对开销进行控制，SVM 模型的预测耗时较少，而且 Macro-F1 值比其他对比算法更好，K 近邻在预测阶段要计算样本距离，耗时明显偏高，综合模型效果、预测开销和工程接入复杂度，本文选择 SVM 部署到 AI 选座模块。实际/api/assistant/seat 接口也包括业务过滤、特征生成和结果排序，接口总耗时应当参照表 6-8 的性能测试结果来确定。

本文对比了多个模型的横向性能，调整了训练集和测试集的划分比例，校验了各个分类标签的预测效果，并且测试了 Top-N 推荐任务，统计了算法预测所耗费的时间，分析表明，支持向量机在目标数据集上的运行表现最为稳定，本文决定把 SVM 作为 AI 辅助选座系统的中心预测模型，通过后端接口将其接入实际选座推荐业务流程，该模型先根据用户需求文本和候选座位特征计算出四类匹配度概率，接着利用预设业务规则、用户偏好匹配情况、冲突项校验规则，生成具有可解释性的座位推荐结果。

6.6 本章小结

本章对系统功能、业务规则、接口性能和推荐算法进行了测试与分析。功能测试表明，小程序端能够完成首页浏览、座位筛选、预约创建、签到签退、暂离返座和记录查询等主要操作，管理后台能够完成公告维护、预约处理、用户信用、座位资源和规则配置等工作；规则与安全性测试表明，系统能够对未登录访问、后台越权、非法参数、同座位冲突和同账号重复预约等情况进行拦截。接口性能测试中，普通业务接口平均响应时间均低于 100 ms，AI 选座接口平均响应时间为 2420 ms，说明推荐接口已经接入真实模型推理流程，但后续仍有优化空间。

推荐实验部分使用 56 条需求和 1772 个座位生成 99232 条需求-座位样本，均衡后保留 34216 条样本，并提取 68 项结构化特征。本文比较了逻辑回归、SVM、MLP、随机森林、决策树、K 近邻和高斯朴素贝叶斯等模型。主实验中，SVM 的 Macro-F1 为 0.9065、Micro-F1 为 0.9269，Top-5、Top-10 和 Top-20 推荐精度分别为 0.9636、0.9273 和 0.8864，说明该模型在本文数据集上具有较好的多标签识别和排序能力。

综合测试结果说明，系统已经包括图书馆座位预约的主要流程，能够把读者端预约、后端规则校验、后台资源治理和 AI 推荐结果连成一个相对完整的原型系统，AI 选座模块可以把自然语言需求变成座位匹配任务，并利用 SVM 输出推荐结果和理由，实际接口耗时仍然比普通查询和预约接口高很多，后续工作可以在模型常驻、候选集裁剪和推荐结果缓存等方面进行改进。

总　结

本文以高校图书馆座位管理场景为研究背景，围绕公平预约、规范管理与智能推荐三个目标，完成了“图书馆智能预约占座系统”微信小程序的设计与实现。系统从读者实际选座和管理员日常维护两类需求出发，完成了总体架构、功能模块、数据库结构和关键业务流程设计，实现了读者端登录、分区浏览、座位筛选、预约创建、签到签退、暂离返座和预约记录查询等功能，也实现了管理员端公告管理、预约处理、用户信用维护、座位状态管理、预约规则配置和运营统计等功能，能够支撑图书馆座位预约从前台使用到后台治理的主要业务流程。

在智能推荐上，本文把自然语言座位需求和空间资源属性结合起来，形成了需求座位多标签匹配任务，实验利用 56 条代表性自然语言需求和 1772 个候选座位生成 99232 条需求与座位匹配样本，并在经过均衡处理的 34216 条样本上提取 68 项结构化特征，把深度专注匹配、舒适久学匹配、设备作业匹配及开放协作四类标签纳入到统一建模过程中。本文对比了逻辑回归、支持向量机 SVM、神经网络 MLP、随机森林、决策树、K 近邻和高斯朴素贝叶斯等模型，实验结果说明，SVM 在主实验中获得 0.9065 的 Macro-F1 和 0.9269 的 Micro-F1，Top-5、Top-10 和 Top-20 推荐精度分别达到 0.9636、0.9273 和 0.8864。模型预测测试表明 SVM 在离线预测阶段花费较少，但是实际 AI 选座接口还需要增加候选过滤、运行时特征构造和推荐排序，所以接口总响应时间超过单纯模型预测所用时间，综合考虑模型效果和工程接入复杂程度，本文最终确定使用 SVM 作为 AI 辅助选座模块的部署模型。

综合系统功能测试、接口联调、并发测试和算法实验结果，本文系统已经具备较完整的业务流程支撑能力和原型级在线推荐能力。读者端能够完成从需求输入到候选座位推荐、预约提交和状态反馈的操作过程，后台端也能对资源、规则和用户状态进行维护，机器学习模型则通过后端推荐接口进入实际业务流程，为自然语言选座需求提供可解释的候选座位结果。但本文工作仍存在继续完善的空间，例如推荐模型主要依赖座位空间属性和规则化需求特征，用户长期偏好、实时人流变化、签到图像校验和更细粒度的语义理解还没有充分纳入系统；同时，AI 选座接口的在线响应速度仍有优化余地。后续可以在扩大真实使用数据规模的基础上，继续优化特征构造、模型部署方式和推荐排序策略，并逐步提升系统的个性化服务能力和实际推广价值。

参考文献

[1] 魏鲁原, 姜雨辰. 高校图书馆座位管理系统[J]. 电子测试, 2021(2): 34-36.

[2] 李昊楠, 刘雅莉. 基于微信小程序的图书馆座位预约系统设计[J]. 现代信息科技, 2020, 4(6): 4-6.

[3] 潘俊芳, 侯振兴. 基于微信小程序的图书馆座位预约系统的设计[J]. 黑龙江工业学院学报(综合版), 2022, 22(5): 66-71.

[4] 向昌盛. 基于微信小程序的图书馆座位预约系统的设计[J]. 国际计算机科学进展, 2022, 2(1): 11-13.

[5] 梁浩, 刘明良, 任爽, 等. 基于微信小程序的图书馆座位预约系统的设计与实现[J]. 计算机科学与应用, 2023, 13(2): 226-232.

[6] Jing Y, Mao L, Xu J. Research on a library seat management system[C]//2019 2nd International Conference on Information Systems and Computer Aided Education. Dalian: IEEE, 2019: 297-299.

[7] Guo W, Liu F, Tian H, *et al.* A novel intelligent management system of library seat[C]//2020 International Conference on Robots & Intelligent System. Sanya: IEEE, 2020: 232-234.

[8] Xu X, Guo Z M, Zhu Y R. Design of seat management system in smart library[C]//2022 10th International Conference on Orange Technology. Shanghai: IEEE, 2022: 1-3.

[9] Adomavicius G, Tuzhilin A. Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions[J]. IEEE Transactions on Knowledge and Data Engineering, 2005, 17(6): 743-749.

[10] Breiman L. Random forests[J]. Machine Learning, 2001, 45(1): 5-6.

[11] Cortes C, Vapnik V. Support-vector networks[J]. Machine Learning, 1995, 20(3): 273-297.

[12] Cover T M, Hart P E. Nearest neighbor pattern classification[J]. IEEE Transactions on Information Theory, 1967, 13(1): 21-27.

[13] Quinlan J R. Induction of decision trees[J]. Machine Learning, 1986, 1(1): 81-106.

[14] Rumelhart D E, Hinton G E, Williams R J. Learning representations by back-propagating errors[J]. Nature, 1986, 323(6088): 533-536.

[15] Arik S O, Pfister T. TabNet: Attentive interpretable tabular learning[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 2021, 35(8): 6679-6687.

[16] Gorishniy Y, Rubachev I, Khrulkov V, *et al.* Revisiting deep learning models for tabular data[C]//Advances in Neural Information Processing Systems. 2021, 34: 18932-18943.

[17] Borisov V, Leemann T, Seßler K, *et al.* Deep neural networks and tabular data: A survey[J]. IEEE Transactions on Neural Networks and Learning Systems, 2024, 35(6): 7499-7519.

[18] Grinsztajn L, Oyallon E, Varoquaux G. Why do tree-based models still outperform deep learning on typical tabular data?[C]//Advances in Neural Information Processing Systems. 2022, 35: 507-520.

[19] 张海藩, 牟永敏. 软件工程导论(第 6 版)[M]. 北京: 清华大学出版社, 2013: 56.

[20] 中华人民共和国国家质量监督检验检疫总局, 中国国家标准化管理委员会. GB/T 25000.10-2016. 系统与软件工程 系统与软件质量要求和评价(SQuaRE) 第 10 部分: 系统与软件质量模型[S]. 北京: 中国标准出版社, 2016.

[21] 王珊, 萨师煊. 数据库系统概论(第 5 版)[M]. 北京: 高等教育出版社, 2014: 214-216.

[22] Zhang M L, Zhou Z H. A review on multi-label learning algorithms[J]. IEEE Transactions on Knowledge and Data Engineering, 2014, 26(8): 1819-1837.

[23] Han J, Kamber M, Pei J. Data mining: concepts and techniques[M]. 3rd ed. Waltham: Morgan Kaufmann, 2012: 112.

[24] 李航. 统计学习方法(第 2 版)[M]. 北京: 清华大学出版社, 2019: 19-21.

[25] 周志华. 机器学习[M]. 北京: 清华大学出版社, 2016: 24-26.

致　谢

四年的本科生活即将结束，毕业论文的完成也为这一阶段的学习画上了句号。在本次毕业设计和论文写作过程中，我得到了老师、组内成员和同学朋友的许多帮助，也在不断尝试和调整中完成了自己的课题，在此表示衷心的感谢。

首先，感谢我的毕业设计指导教师谢润山老师。在选题确定、系统方案梳理、实验思路调整和论文修改过程中，谢老师给予了我耐心细致的指导，使我能够不断发现问题、修正思路并稳步推进。本文选题涉及微信小程序设计、后端接口开发和深度学习相关实验测试等内容，这些方向都是我较为感兴趣的方面，因此整个毕业设计过程也让我能够更加投入地完成系统开发和论文整理。通过本次课题实践，我不仅加深了对小程序开发、数据库设计和推荐模型实验的理解，也更加体会到将兴趣转化为实际成果的意义。

其次，感谢课题组内其他成员在毕业设计过程中给予我的帮助。在系统开发、资料查找、问题咨询、论文修改和实验调整等方面，大家都给予了我许多建议和支持，使我在遇到问题时能够更快地梳理思路、找到解决方向。同时，也感谢在项目讨论、系统联调和论文交流过程中给予我帮助的同学与朋友，你们的鼓励和协助让我顺利推进了本次毕业设计。

最后，也感谢在这段时间里始终坚持完成任务的自己。面对系统实现、实验调整和论文反复修改中的问题，我学会了更加独立地分析和解决困难，也逐渐培养了不轻易退缩的态度。未来我将继续保持对技术学习的兴趣，在新的阶段不断积累经验、提升自己。谨以此文，向所有帮助和支持过我的老师、组内成员、同学与朋友致以诚挚的感谢。